

SemZip Evidence Supplement

Anonymous research artifact

October 2026

Scope. This supplement preserves completed historical evidence and separates it from the cost-guided, first-block-trained, guarded main comparison. The single-synthesis deployment (SemZip-1) and its external comparison are retained as the ablation reference. The matched-representation controls, first-block training, the 252-trial serialized timing session, and the online-evolution campaign are complete. No historical ratio or failure denominator is replaced with a later outcome.

1 Main-cohort result and provenance interfaces

The current main method uses five first-block syntheses (one greedy, four at temperature 0.7), cost-guided selection on block 0, a frozen storage policy, and the actual semantic-archive guard. SemZip-1 deploys the greedy synthesis alone. Each of the sixteen original files retains its byte identity and 100,000-LF-record blocks. Tables are generated from audited ledgers. An unavailable or failed observation is not a zero or a measured ratio. The selected deployment’s sixteen complete-file observations all pass archive-only decoding and the independent block audit, and its throughput cells come from one serialized session together with SemZip-1, DeLog, LogLite-BL, gzip6, XZ6 and Zstd3.

1.1 Complete-file outcomes of the selected deployment

Table 1. Complete-file outcomes of the selected deployment. Suffix covers original blocks 1 onward with the recomputed suffix manifest (SemZip-1 accounting), with – for one-block files. Recov. counts blocks that took the predetermined recovery. Every file passed archive-only decoding and an independent materialized-file block audit.

Dataset	Blocks	Raw bytes	Archive bytes	Ratio	Suffix ratio	Recov.	SemZip-1
Linux	1	2,349,686	58,613	40.09	–	0	38.47
Proxifier	1	2,541,814	70,361	36.13	–	0	34.56
Apache	1	5,135,876	69,646	73.74	–	0	72.05
Zookeeper	1	10,426,438	55,045	189.42	–	0	179.00
Mac	2	16,879,552	311,853	54.13	33.14	0	49.03
HealthApp	3	23,529,930	395,273	59.53	57.35	0	38.49
HPC	5	33,553,503	732,843	45.79	45.03	0	31.62
Hadoop	4	48,595,595	571,500	85.03	109.25	0	84.53
OpenStack	3	61,442,082	2,434,257	25.24	26.29	0	25.10
OpenSSH	7	73,417,506	623,323	117.78	113.95	0	115.34
Android	16	192,270,829	5,565,790	34.55	34.77	0	34.54
BGL	48	743,185,031	14,203,694	52.32	52.40	0	40.60
HDFS	112	1,577,982,906	50,818,812	31.05	31.00	0	27.20
Spark	333	2,941,224,304	47,170,447	62.35	62.41	0	56.20
Windows	1147	28,012,696,901	45,285,306	618.58	618.69	0	530.10
Thunderbird	2113	31,788,301,041	490,102,034	64.86	64.87	0	64.65

1.2 Formal throughput cells of the selected deployment and baselines

Table 2. Formal throughput cells (decimal MB/s) give the median and range of three clean serialized trials per file and method. A trial was repeated when another workload in the same eight-CPU allocation used more than 0.5 cores during its encode or decode phase. Repeated attempts are retained in the artifact.

Dataset	Method	Encode median	Encode range	Decode median	Decode range
Linux	SemZip	0.55	0.54–0.56	1.48	1.48–1.61
	SemZip-1	0.55	0.53–0.58	1.48	1.48–1.63
	DeLog	1.98	1.91–2.05	2.26	2.10–2.27
	LogLite-BL*	1.99	1.99–2.15	2.46	2.44–2.53
	gzip6	3.10	3.05–3.10	3.04	2.89–3.09
	XZ6	1.74	1.71–1.84	2.81	2.81–2.89
	Zstd3	3.14	2.96–3.17	2.88	2.81–3.07
Proxifier	SemZip	0.56	0.49–0.56	1.59	1.28–1.59
	SemZip-1	0.51	0.51–0.52	1.48	1.42–1.48
	DeLog	2.20	2.13–2.20	2.26	2.24–2.40
	LogLite-BL*	2.27	2.19–2.31	2.25	2.14–2.58
	gzip6	3.15	3.11–3.19	2.93	2.24–3.30
	XZ6	2.15	1.90–2.21	3.02	2.14–3.08
	Zstd3	3.17	2.66–3.19	3.00	2.18–3.12
Apache	SemZip	0.93	0.76–0.97	2.64	2.52–2.98
	SemZip-1	0.96	0.78–0.99	2.71	2.35–3.20
	DeLog	3.58	2.92–3.73	3.42	2.89–4.89
	LogLite-BL*	3.59	2.93–3.69	3.04	2.92–3.44
	gzip6	4.74	3.99–5.21	4.11	2.95–4.38
	XZ6	2.94	2.54–3.03	5.24	3.03–5.43
	Zstd3	4.96	4.06–5.17	4.38	4.18–4.47
Zookeeper	SemZip	0.72	0.71–0.74	2.90	2.80–3.54
	SemZip-1	0.71	0.70–0.73	2.83	2.67–3.20
	DeLog	4.73	4.63–4.85	7.30	6.64–7.72
	LogLite-BL*	5.19	4.25–5.43	6.82	6.71–7.17
	gzip6	7.13	5.33–7.32	12.63	8.01–13.70
	XZ6	2.90	2.89–3.22	9.30	7.45–9.44
	Zstd3	5.96	5.46–7.36	11.86	10.47–14.53
Mac	SemZip	1.35	1.32–1.37	5.39	4.85–5.71
	SemZip-1	1.10	1.09–1.14	4.70	4.55–4.73
	DeLog	5.93	5.73–6.07	13.04	12.99–13.48
	LogLite-BL*	7.03	6.35–7.87	17.54	14.75–18.26
	gzip6	10.16	10.00–13.09	26.75	16.56–27.76
	XZ6	4.16	4.03–4.81	14.79	11.08–16.51
	Zstd3	10.32	10.21–10.77	27.38	17.15–28.02
HealthApp	SemZip	1.49	1.46–1.50	9.24	8.84–9.55
	SemZip-1	3.58	3.42–3.58	12.83	12.43–12.85
	DeLog	10.30	10.06–11.17	20.70	20.15–23.87
	LogLite-BL*	9.54	9.03–10.85	24.37	21.56–28.60
	gzip6	15.16	12.55–18.76	36.33	35.52–36.77
	XZ6	5.51	5.48–5.95	23.90	21.90–28.13
	Zstd3	16.81	15.09–17.10	35.91	35.06–37.38
HPC	SemZip	6.50	5.73–6.69	17.00	15.57–18.20
	SemZip-1	2.69	2.61–2.73	11.70	11.14–11.89
	DeLog	13.26	11.97–14.03	22.58	22.54–23.39
	LogLite-BL*	14.11	11.88–14.55	25.95	25.08–27.57

Table 2. (continued)

Dataset	Method	Encode median	Encode range	Decode median	Decode range
	gzip6	23.68	22.81–25.59	43.21	42.29–45.35
	XZ6	6.48	6.45–6.61	32.09	29.41–34.41
	Zstd3	24.17	21.76–27.21	36.27	36.22–43.44
Hadoop	SemZip	2.83	2.67–2.89	13.42	13.29–13.43
	SemZip-1	2.86	2.77–2.90	14.10	13.52–14.71
	DeLog	20.72	19.95–21.64	40.75	37.13–43.87
	LogLite-BL*	19.42	19.37–20.94	43.08	42.45–48.56
	gzip6	35.11	32.70–35.68	64.64	64.35–69.11
	XZ6	13.10	11.69–13.42	51.28	48.51–53.47
	Zstd3	38.07	37.41–40.47	64.86	62.39–67.65
OpenStack	SemZip	2.53	2.48–2.54	18.31	17.44–18.86
	SemZip-1	2.26	2.14–2.27	13.84	13.60–14.24
	DeLog	10.83	10.38–11.08	36.37	36.22–40.43
	LogLite-BL*	14.12	13.47–15.49	41.15	36.08–43.43
	gzip6	37.19	34.41–38.22	61.37	61.12–71.47
	XZ6	7.75	7.50–7.95	40.83	37.00–52.82
	Zstd3	43.59	41.31–45.52	82.06	79.42–83.73
OpenSSH	SemZip	1.98	1.94–1.98	10.83	10.65–10.91
	SemZip-1	2.11	2.10–2.14	12.37	12.24–12.69
	DeLog	21.82	20.56–25.24	43.64	40.61–45.00
	LogLite-BL*	23.55	23.24–24.38	50.73	48.06–50.86
	gzip6	43.54	40.54–47.02	77.17	68.60–84.49
	XZ6	10.49	10.22–11.08	58.25	56.85–64.00
	Zstd3	44.81	44.23–47.11	80.76	67.19–82.51
Android	SemZip	6.22	6.02–6.42	30.33	29.95–32.94
	SemZip-1	6.29	6.22–6.51	32.32	32.05–32.56
	DeLog	23.83	22.84–25.05	56.03	50.85–59.07
	LogLite-BL*	20.77	19.34–21.34	60.44	58.97–69.92
	gzip6	66.33	60.33–68.84	128.98	122.04–129.03
	XZ6	15.69	14.75–16.30	101.54	99.43–107.50
	Zstd3	92.80	84.14–94.36	136.30	124.65–138.48
BGL	SemZip	2.70	2.68–2.72	21.17	20.63–21.22
	SemZip-1	5.67	5.59–5.71	42.50	42.00–42.90
	DeLog	30.40	30.12–30.89	78.33	76.84–78.66
	LogLite-BL*	32.33	32.24–33.94	100.27	96.75–104.04
	gzip6	99.44	98.06–104.41	177.21	163.53–189.29
	XZ6	10.87	10.64–11.26	151.01	144.55–151.13
	Zstd3	155.42	154.00–164.71	202.18	200.79–210.74

16 attempts were repeated because of concurrent load, and no clean observation was discarded.

1.3 SemZip-1 cohort records (ablation reference)

The records below belong to the single-synthesis SemZip-1 cohort and the external first passes of the earlier campaign. Throughout Sections 1.3.1–1.3.7, SemZip/semzip denotes SemZip-1. All 96 of its complete-file observations and 216 earlier formal trials passed, and repeated archives matched their accepted first-pass bytes. DeLog, LogLite-BL and gzip6 archive sizes in the main table, and XZ6 and Zstd3 sizes in Table 24, come from these first passes, and the other baselines from Section 13. Small-file throughputs in the main text come from the serialized twelve-file session, and large-file throughputs from a separate twenty-block session (Table 32).

Table 3. Nontraining suffix archive-cost view on the fixed 12 original multi-block files. Every method uses the same original blocks 1 onward. SemZip-1 (single-synthesis cohort). Ratios count SemZip-1's reconstructed suffix manifest and each external method's existing independent archive files. No suffix is recompressed or separately timed. One-block files are excluded. Failed observations receive no ratio or reduced-denominator aggregate. *LogLite-BL-wide+tail+reservefix.

Dataset	SemZip-1	DeLog	LogLite-BL*	gzip6	XZ6	Zstd3
Mac	31.42	26.77	18.97	10.10	20.41	15.23
HealthApp	37.20	48.62	16.44	10.27	15.76	9.89
HPC	30.50	43.80	19.32	9.92	18.89	9.33
Hadoop	108.90	105.27	50.72	25.43	52.26	36.37
OpenStack	26.72	22.95	17.40	11.93	16.40	12.76
OpenSSH	111.50	104.48	32.58	15.93	21.32	14.21
Android	34.77	29.55	17.06	7.56	19.45	12.62
BGL	40.72	40.42	24.19	11.86	22.07	10.62
HDFS	27.16	29.79	12.66	10.13	15.62	10.27
Spark	56.23	59.11	37.13	16.06	23.08	15.26
Windows	530.16	536.26	300.51	16.77	217.86	146.01
Thunderbird	64.66	67.69	39.78	15.76	28.54	18.83
Arithmetic mean (12)	91.66	92.89	48.90	13.48	39.31	25.95
Total suffix raw / archive	96.04	100.35	57.23	15.86	43.01	28.18

1.3.1 Nontraining suffix comparison

The suffix comparison retains the same 12 multi-block original files for all six methods and excludes 4 one-block files. SemZip-1 has smaller suffix archives than DeLog on 6/12 jointly valid files. Their respective arithmetic means are $91.66\times$ and $92.89\times$, and corpus ratios are $96.04\times$ and $100.35\times$. SemZip-1's summed suffix archive bytes are 4.48% larger than DeLog. The suffix and sixteen-file means use different dataset cohorts, so their difference does not isolate the effect of removing training blocks. This view uses existing independently checked block payloads, with SemZip-1's suffix manifest counted. It has no separate suffix timing, and matched-representation suffix savings instead compare payloads only.

Table 4. Suffix costs in bytes. Payload is the sum of existing selected block archive files. SemZip additionally counts its reconstructed suffix manifest. A failure is a recorded terminal outcome, not a zero.

Dataset	Method	Raw	Payload	Manifest	Archive	Status
Mac	SemZip	2,460,682	78,172	151	78,323	PASS
Mac	DeLog	2,460,682	91,916	0	91,916	PASS
Mac	LogLite-BL*	2,460,682	129,741	0	129,741	PASS
Mac	gzip6	2,460,682	243,730	0	243,730	PASS
Mac	XZ6	2,460,682	120,580	0	120,580	PASS
Mac	Zstd3	2,460,682	161,530	0	161,530	PASS
HealthApp	SemZip	14,186,163	381,188	187	381,375	PASS
HealthApp	DeLog	14,186,163	291,788	0	291,788	PASS
HealthApp	LogLite-BL*	14,186,163	862,774	0	862,774	PASS
HealthApp	gzip6	14,186,163	1,381,701	0	1,381,701	PASS
HealthApp	XZ6	14,186,163	900,096	0	900,096	PASS
HealthApp	Zstd3	14,186,163	1,433,872	0	1,433,872	PASS
HPC	SemZip	24,429,742	800,708	241	800,949	PASS
HPC	DeLog	24,429,742	557,788	0	557,788	PASS
HPC	LogLite-BL*	24,429,742	1,264,212	0	1,264,212	PASS
HPC	gzip6	24,429,742	2,461,564	0	2,461,564	PASS
HPC	XZ6	24,429,742	1,293,164	0	1,293,164	PASS
HPC	Zstd3	24,429,742	2,617,926	0	2,617,926	PASS
Hadoop	SemZip	31,409,632	288,216	214	288,430	PASS
Hadoop	DeLog	31,409,632	298,376	0	298,376	PASS
Hadoop	LogLite-BL*	31,409,632	619,263	0	619,263	PASS
Hadoop	gzip6	31,409,632	1,235,247	0	1,235,247	PASS

Table 4. Suffix costs in bytes. Payload is the sum of existing selected block archive files. SemZip additionally counts its reconstructed suffix manifest. A failure is a recorded terminal outcome, not a zero.

Dataset	Method	Raw	Payload	Manifest	Archive	Status
Hadoop	XZ6	31,409,632	601,076	0	601,076	PASS
Hadoop	Zstd3	31,409,632	863,622	0	863,622	PASS
OpenStack	SemZip	31,838,221	1,191,436	187	1,191,623	PASS
OpenStack	DeLog	31,838,221	1,387,556	0	1,387,556	PASS
OpenStack	LogLite-BL*	31,838,221	1,829,578	0	1,829,578	PASS
OpenStack	gzip6	31,838,221	2,668,560	0	2,668,560	PASS
OpenStack	XZ6	31,838,221	1,940,844	0	1,940,844	PASS
OpenStack	Zstd3	31,838,221	2,495,098	0	2,495,098	PASS
OpenSSH	SemZip	62,372,265	559,092	305	559,397	PASS
OpenSSH	DeLog	62,372,265	596,996	0	596,996	PASS
OpenSSH	LogLite-BL*	62,372,265	1,914,198	0	1,914,198	PASS
OpenSSH	gzip6	62,372,265	3,914,797	0	3,914,797	PASS
OpenSSH	XZ6	62,372,265	2,925,996	0	2,925,996	PASS
OpenSSH	Zstd3	62,372,265	4,389,369	0	4,389,369	PASS
Android	SemZip	180,436,054	5,188,848	576	5,189,424	PASS
Android	DeLog	180,436,054	6,106,772	0	6,106,772	PASS
Android	LogLite-BL*	180,436,054	10,578,891	0	10,578,891	PASS
Android	gzip6	180,436,054	23,882,086	0	23,882,086	PASS
Android	XZ6	180,436,054	9,278,120	0	9,278,120	PASS
Android	Zstd3	180,436,054	14,292,671	0	14,292,671	PASS
BGL	SemZip	729,925,313	17,926,076	1,532	17,927,608	PASS
BGL	DeLog	729,925,313	18,058,428	0	18,058,428	PASS
BGL	LogLite-BL*	729,925,313	30,171,647	0	30,171,647	PASS
BGL	gzip6	729,925,313	61,522,499	0	61,522,499	PASS
BGL	XZ6	729,925,313	33,066,880	0	33,066,880	PASS
BGL	Zstd3	729,925,313	68,715,720	0	68,715,720	PASS
HDFS	SemZip	1,563,882,073	57,568,668	3,454	57,572,122	PASS
HDFS	DeLog	1,563,882,073	52,503,700	0	52,503,700	PASS
HDFS	LogLite-BL*	1,563,882,073	123,546,831	0	123,546,831	PASS
HDFS	gzip6	1,563,882,073	154,457,366	0	154,457,366	PASS
HDFS	XZ6	1,563,882,073	100,089,192	0	100,089,192	PASS
HDFS	Zstd3	1,563,882,073	152,308,892	0	152,308,892	PASS
Spark	SemZip	2,929,405,027	52,083,788	10,085	52,093,873	PASS
Spark	DeLog	2,929,405,027	49,557,120	0	49,557,120	PASS
Spark	LogLite-BL*	2,929,405,027	78,895,676	0	78,895,676	PASS
Spark	gzip6	2,929,405,027	182,455,860	0	182,455,860	PASS
Spark	XZ6	2,929,405,027	126,899,540	0	126,899,540	PASS
Spark	Zstd3	2,929,405,027	192,019,266	0	192,019,266	PASS
Windows	SemZip	27,990,612,658	52,762,484	34,508	52,796,992	PASS
Windows	DeLog	27,990,612,658	52,195,876	0	52,195,876	PASS
Windows	LogLite-BL*	27,990,612,658	93,142,674	0	93,142,674	PASS
Windows	gzip6	27,990,612,658	1,669,187,392	0	1,669,187,392	PASS
Windows	XZ6	27,990,612,658	128,477,228	0	128,477,228	PASS
Windows	Zstd3	27,990,612,658	191,708,267	0	191,708,267	PASS
Thunderbird	SemZip	31,773,767,445	491,315,816	63,492	491,379,308	PASS
Thunderbird	DeLog	31,773,767,445	469,426,908	0	469,426,908	PASS
Thunderbird	LogLite-BL*	31,773,767,445	798,649,136	0	798,649,136	PASS
Thunderbird	gzip6	31,773,767,445	2,016,276,752	0	2,016,276,752	PASS
Thunderbird	XZ6	31,773,767,445	1,113,326,532	0	1,113,326,532	PASS
Thunderbird	Zstd3	31,773,767,445	1,687,688,213	0	1,687,688,213	PASS

1.3.2 Complete-file outcomes and component costs

Table 5. Unsuccessful complete-file observations in the earlier six-method (SemZip-1) campaign. LogShrink and LogReducer failures are in Section 13. A generic failure status does not establish a reconstruction mismatch.

Dataset	Method	Terminal status	Recorded stage
No unsuccessful complete-file observations.			

Table 6. Complete-file deployment components. Semantic, residual, and manifest bytes sum to the complete archive. Original input byte counts use the unchanged source files.

Dataset	Raw bytes	Semantic B	Residual B	Manifest B
Linux	2,349,686	30,752	30,180	153
Proxifier	2,541,814	65,296	8,100	157
Apache	5,135,876	45,760	25,368	154
Zookeeper	10,426,438	46,804	11,288	157
Mac	16,879,552	71,132	272,944	181
HealthApp	23,529,930	83,644	527,532	217
HPC	33,553,503	868,492	192,292	271
Hadoop	48,595,595	198,944	375,720	244
OpenStack	61,442,082	1,970,888	476,312	217
OpenSSH	73,417,506	575,140	61,052	335
Android	192,270,829	525,516	5,039,724	606
BGL	743,185,031	9,069,748	9,233,648	1,562
HDFS	1,577,982,906	27,938,388	30,070,200	3,484
Spark	2,941,224,304	13,117,972	39,206,508	10,115
Windows	28,012,696,901	30,861,340	21,947,864	34,538
Thunderbird	31,788,301,041	777,584	490,831,268	63,522

Table 7. Deployment activity and suffix accounting. Active rules have at least one stored value. Suffix ratios use original blocks 1 onward with a recomputed sequential manifest and are not separately timed. One-block files have no suffix.

Dataset	Blocks	Recovered blocks	Active/saved rules	Suffix ratio
Linux	1	0	8/8	—
Proxifier	1	0	6/6	—
Apache	1	0	3/3	—
Zookeeper	1	0	6/7	—
Mac	2	0	7/8	31.42
HealthApp	3	0	1/1	37.20
HPC	5	0	3/4	30.50
Hadoop	4	0	6/9	108.90
OpenStack	3	0	8/19	26.72
OpenSSH	7	0	6/6	111.50
Android	16	0	5/5	34.77
BGL	48	0	2/7	40.72
HDFS	112	0	1/3	27.16
Spark	333	0	16/18	56.23
Windows	1,147	0	12/16	530.16

Table 7. Deployment activity and suffix accounting. Active rules have at least one stored value. Suffix ratios use original blocks 1 onward with a recomputed sequential manifest and are not separately timed. One-block files have no suffix.

Dataset	Blocks	Recovered blocks	Active/saved rules	Suffix ratio
Thunderbird	2,113	0	0/2	64.66

1.3.3 All formal throughput cells

Table 8. Formal per-file throughput medians [minimum, maximum], decimal MB/s. A median requires three successful identical archives that match the accepted first pass. All three attempt statuses are retained.

Dataset	Method	Encode	Decode	Cell / attempt statuses
Linux	SemZip	0.52 [0.52, 0.62]	1.21 [1.21, 1.73]	PASS/PASS (3)
Linux	DeLog	1.84 [1.72, 1.84]	1.81 [1.79, 2.64]	PASS/PASS (3)
Linux	LogLite-BL*	1.95 [1.82, 2.50]	2.22 [2.08, 3.53]	PASS/PASS (3)
Linux	gzip6	3.01 [2.56, 4.08]	2.82 [2.41, 4.16]	PASS/PASS (3)
Linux	XZ6	1.69 [1.55, 2.02]	2.49 [2.30, 4.06]	PASS/PASS (3)
Linux	Zstd3	3.28 [1.50, 3.89]	2.67 [2.37, 3.96]	PASS/PASS (3)
Proxifier	SemZip	0.49 [0.48, 0.57]	1.19 [1.01, 1.70]	PASS/PASS (3)
Proxifier	DeLog	2.08 [1.94, 2.60]	1.83 [1.76, 2.94]	PASS/PASS (3)
Proxifier	LogLite-BL*	2.21 [2.19, 2.86]	2.34 [2.05, 3.58]	PASS/PASS (3)
Proxifier	gzip6	3.87 [2.90, 4.37]	2.62 [2.38, 4.57]	PASS/PASS (3)
Proxifier	XZ6	2.06 [1.87, 2.61]	2.57 [2.39, 4.50]	PASS/PASS (3)
Proxifier	Zstd3	3.55 [2.66, 4.53]	2.62 [2.37, 4.55]	PASS/PASS (3)
Apache	SemZip	0.86 [0.77, 1.16]	2.05 [1.87, 3.66]	PASS/PASS (3)
Apache	DeLog	3.06 [2.80, 4.62]	3.03 [2.95, 5.93]	PASS/PASS (3)
Apache	LogLite-BL*	3.27 [3.12, 4.81]	4.61 [2.53, 6.62]	PASS/PASS (3)
Apache	gzip6	4.55 [4.34, 8.29]	6.44 [6.12, 9.07]	PASS/PASS (3)
Apache	XZ6	2.67 [2.50, 3.78]	3.55 [3.53, 8.41]	PASS/PASS (3)
Apache	Zstd3	7.04 [4.63, 8.99]	5.74 [3.78, 9.67]	PASS/PASS (3)
Zookeeper	SemZip	0.76 [0.75, 0.88]	3.04 [2.81, 3.81]	PASS/PASS (3)
Zookeeper	DeLog	5.07 [4.87, 6.71]	6.50 [6.28, 10.22]	PASS/PASS (3)
Zookeeper	LogLite-BL*	5.57 [5.48, 8.89]	6.99 [6.80, 13.90]	PASS/PASS (3)
Zookeeper	gzip6	8.37 [7.90, 15.78]	14.06 [13.73, 17.03]	PASS/PASS (3)
Zookeeper	XZ6	3.58 [3.58, 4.60]	8.30 [8.02, 16.39]	PASS/PASS (3)
Zookeeper	Zstd3	8.74 [7.97, 15.83]	15.05 [14.41, 16.54]	PASS/PASS (3)
Mac	SemZip	1.19 [1.15, 1.30]	4.47 [4.26, 5.21]	PASS/PASS (3)
Mac	DeLog	6.26 [6.13, 8.28]	11.00 [10.98, 14.42]	PASS/PASS (3)
Mac	LogLite-BL*	8.00 [6.63, 9.35]	15.66 [15.16, 19.52]	PASS/PASS (3)
Mac	gzip6	13.01 [12.45, 21.98]	21.93 [19.71, 25.50]	PASS/PASS (3)
Mac	XZ6	5.16 [5.10, 5.76]	14.74 [14.43, 23.21]	PASS/PASS (3)
Mac	Zstd3	20.98 [18.71, 24.23]	16.46 [15.94, 25.92]	PASS/PASS (3)
HealthApp	SemZip	3.59 [3.56, 4.03]	10.59 [10.48, 13.35]	PASS/PASS (3)
HealthApp	DeLog	12.14 [12.05, 14.55]	16.81 [16.52, 20.49]	PASS/PASS (3)
HealthApp	LogLite-BL*	11.32 [11.18, 13.95]	18.78 [18.18, 25.51]	PASS/PASS (3)
HealthApp	gzip6	20.56 [17.17, 27.55]	30.06 [29.97, 35.39]	PASS/PASS (3)
HealthApp	XZ6	6.26 [5.99, 6.59]	22.87 [20.61, 32.35]	PASS/PASS (3)
HealthApp	Zstd3	21.67 [19.86, 35.21]	29.85 [29.83, 35.93]	PASS/PASS (3)
HPC	SemZip	2.71 [2.70, 2.93]	10.20 [9.53, 11.24]	PASS/PASS (3)
HPC	DeLog	13.07 [12.41, 15.53]	19.25 [18.68, 21.27]	PASS/PASS (3)
HPC	LogLite-BL*	14.22 [14.19, 15.89]	23.38 [21.67, 26.02]	PASS/PASS (3)
HPC	gzip6	27.43 [24.57, 28.86]	37.16 [29.56, 43.26]	PASS/PASS (3)

Table 8. Formal per-file throughput medians [minimum, maximum], decimal MB/s. A median requires three successful identical archives that match the accepted first pass. All three attempt statuses are retained.

Dataset	Method	Encode	Decode	Cell / attempt statuses
HPC	XZ6	6.49 [6.41, 7.29]	28.57 [26.32, 35.33]	PASS/PASS (3)
HPC	Zstd3	34.69 [25.08, 42.03]	36.17 [30.93, 43.73]	PASS/PASS (3)
Hadoop	SemZip	2.94 [2.91, 2.95]	12.52 [12.30, 12.66]	PASS/PASS (3)
Hadoop	DeLog	21.16 [20.06, 23.07]	33.24 [32.50, 41.87]	PASS/PASS (3)
Hadoop	LogLite-BL*	20.33 [19.37, 21.53]	45.75 [42.09, 49.48]	PASS/PASS (3)
Hadoop	gzip6	42.40 [37.57, 48.88]	53.75 [45.96, 58.32]	PASS/PASS (3)
Hadoop	XZ6	13.50 [13.26, 14.46]	44.56 [44.13, 51.79]	PASS/PASS (3)
Hadoop	Zstd3	40.85 [40.00, 55.74]	55.42 [47.99, 58.67]	PASS/PASS (3)
OpenStack	SemZip	2.24 [2.23, 2.33]	13.12 [12.90, 13.72]	PASS/PASS (3)
OpenStack	DeLog	11.14 [11.00, 11.34]	33.58 [28.94, 37.64]	PASS/PASS (3)
OpenStack	LogLite-BL*	14.11 [13.14, 15.00]	38.45 [34.74, 47.40]	PASS/PASS (3)
OpenStack	gzip6	41.23 [33.91, 48.49]	57.80 [56.50, 73.17]	PASS/PASS (3)
OpenStack	XZ6	7.64 [7.62, 8.00]	41.46 [38.07, 44.75]	PASS/PASS (3)
OpenStack	Zstd3	45.77 [35.31, 52.87]	54.58 [52.15, 66.08]	PASS/PASS (3)
OpenSSH	SemZip	2.13 [2.13, 2.14]	11.54 [10.93, 11.76]	PASS/PASS (3)
OpenSSH	DeLog	23.77 [22.72, 26.06]	38.53 [36.84, 38.93]	PASS/PASS (3)
OpenSSH	LogLite-BL*	26.20 [24.79, 26.37]	45.14 [42.26, 46.70]	PASS/PASS (3)
OpenSSH	gzip6	58.84 [48.49, 63.11]	69.61 [64.62, 70.46]	PASS/PASS (3)
OpenSSH	XZ6	10.94 [10.63, 11.60]	56.23 [53.78, 61.02]	PASS/PASS (3)
OpenSSH	Zstd3	66.93 [50.91, 69.39]	64.29 [60.84, 78.56]	PASS/PASS (3)
Android	SemZip	6.35 [6.35, 6.54]	29.68 [28.14, 30.18]	PASS/PASS (3)
Android	DeLog	25.33 [23.06, 26.16]	48.90 [47.98, 50.13]	PASS/PASS (3)
Android	LogLite-BL*	20.26 [17.77, 21.77]	60.93 [57.96, 61.98]	PASS/PASS (3)
Android	gzip6	82.92 [73.88, 83.96]	112.17 [108.41, 124.13]	PASS/PASS (3)
Android	XZ6	15.88 [15.58, 15.97]	103.58 [99.23, 106.29]	PASS/PASS (3)
Android	Zstd3	98.20 [85.41, 114.00]	117.36 [116.28, 135.54]	PASS/PASS (3)
BGL	SemZip	5.79 [5.76, 5.81]	39.59 [39.09, 40.18]	PASS/PASS (3)
BGL	DeLog	29.92 [29.07, 30.00]	71.38 [69.25, 71.66]	PASS/PASS (3)
BGL	LogLite-BL*	32.20 [31.14, 32.48]	91.38 [85.94, 92.39]	PASS/PASS (3)
BGL	gzip6	100.95 [100.33, 108.58]	176.36 [171.69, 181.17]	PASS/PASS (3)
BGL	XZ6	11.01 [10.71, 11.46]	142.17 [140.65, 142.70]	PASS/PASS (3)
BGL	Zstd3	159.08 [147.10, 170.87]	178.57 [159.21, 181.92]	PASS/PASS (3)

1.3.4 Complete first-pass observations

Table 9. All complete-file first-pass archive bytes and descriptive inner-timer seconds. Campaigns may overlap, and their inner timer boundaries differ. These durations are not formal cross-method speed observations.

Dataset	Method	Archive bytes/status	Encode s	Decode s
Linux	semzip	61,085	5.960	1.563
Linux	delog	85,140	0.682	0.648
Linux	loglite	124,029	0.538	0.326
Linux	gzip6	231,916	0.141	0.137
Linux	xz6	117,636	0.742	0.138
Linux	zstd3	191,936	0.194	0.135
Proxifier	semzip	73,553	5.193	1.432
Proxifier	delog	84,180	0.555	0.442
Proxifier	loglite	151,957	0.476	0.360

Table 9. All complete-file first-pass archive bytes and descriptive inner-timer seconds. Campaigns may overlap, and their inner timer boundaries differ. These durations are not formal cross-method speed observations.

Dataset	Method	Archive bytes/status	Encode s	Decode s
Proxifier	gzip6	172,192	0.153	0.136
Proxifier	xz6	120,608	0.541	0.743
Proxifier	zstd3	187,045	0.141	0.135
Apache	semzip	71,282	4.392	1.179
Apache	delog	85,932	0.666	0.500
Apache	loglite	177,981	0.727	0.415
Apache	gzip6	262,804	0.197	0.140
Apache	xz6	180,972	0.929	0.493
Apache	zstd3	272,478	0.147	0.139
Zookeeper	semzip	58,249	12.260	2.498
Zookeeper	delog	66,456	1.027	0.559
Zookeeper	loglite	177,621	0.853	0.330
Zookeeper	gzip6	459,781	0.261	0.150
Zookeeper	xz6	351,892	2.441	0.201
Zookeeper	zstd3	452,767	0.250	0.902
Mac	semzip	344,257	13.288	2.894
Mac	delog	381,676	1.677	0.787
Mac	loglite	680,270	1.791	0.585
Mac	gzip6	1,490,626	0.345	0.217
Mac	xz6	707,444	2.957	0.518
Mac	zstd3	995,882	0.234	0.216
HealthApp	semzip	611,393	5.588	1.404
HealthApp	delog	457,764	1.158	0.653
HealthApp	loglite	1,420,599	1.483	0.579
HealthApp	gzip6	2,297,301	0.319	0.235
HealthApp	xz6	1,486,284	3.137	0.338
HealthApp	zstd3	2,372,463	0.276	0.418
HPC	semzip	1,061,055	11.380	2.389
HPC	delog	750,604	1.946	1.033
HPC	loglite	1,667,169	1.772	0.900
HPC	gzip6	3,290,895	0.480	0.647
HPC	xz6	1,693,708	4.223	0.563
HPC	zstd3	3,428,677	0.381	0.362
Hadoop	semzip	574,908	16.026	3.045
Hadoop	delog	602,948	1.421	0.854
Hadoop	loglite	1,309,820	1.707	0.653
Hadoop	gzip6	2,531,227	0.397	0.335
Hadoop	xz6	1,290,496	2.852	0.478
Hadoop	zstd3	1,857,124	0.488	0.326
OpenStack	semzip	2,447,417	26.974	3.946
OpenStack	delog	2,831,008	5.123	1.117
OpenStack	loglite	3,746,859	4.659	0.967
OpenStack	gzip6	5,395,973	0.944	0.456
OpenStack	xz6	3,925,064	6.881	0.783
OpenStack	zstd3	5,059,821	0.404	0.408
OpenSSH	semzip	636,527	34.010	5.678
OpenSSH	delog	681,308	2.827	1.488
OpenSSH	loglite	2,173,771	2.587	1.063
OpenSSH	gzip6	4,596,730	0.832	0.453
OpenSSH	xz6	3,391,404	6.087	0.711
OpenSSH	zstd3	5,127,161	0.533	0.506
Android	semzip	5,565,846	30.828	5.879

Table 9. All complete-file first-pass archive bytes and descriptive inner-timer seconds. Campaigns may overlap, and their inner timer boundaries differ. These durations are not formal cross-method speed observations.

Dataset	Method	Archive bytes/status	Encode s	Decode s
Android	delog	6,564,784	7.414	3.356
Android	loglite	11,395,252	9.504	2.885
Android	gzip6	25,717,765	1.718	1.113
Android	xz6	9,925,016	11.782	1.321
Android	zstd3	15,310,801	1.313	1.053
BGL	semzip	18,304,958	136.248	18.226
BGL	delog	18,428,912	23.889	9.838
BGL	loglite	30,767,804	25.072	7.632
BGL	gzip6	62,939,897	6.279	3.502
BGL	xz6	33,700,312	66.799	4.681
BGL	zstd3	70,285,108	6.429	3.246
HDFS	semzip	58,012,072	140.755	28.998
HDFS	delog	52,898,088	53.808	21.146
HDFS	loglite	124,571,784	80.923	20.065
HDFS	gzip6	155,729,929	13.056	7.728
HDFS	xz6	100,965,828	153.608	9.835
HDFS	zstd3	153,595,624	8.138	7.346
Spark	semzip	52,334,595	834.136	191.786
Spark	delog	49,805,988	101.717	53.408
Spark	loglite	79,450,009	91.502	42.340
Spark	gzip6	183,444,090	27.299	17.416
Spark	xz6	127,479,660	235.778	21.677
Spark	zstd3	192,937,544	19.988	18.475
Windows	semzip	52,843,742	3158.752	539.626
Windows	delog	52,244,728	610.911	224.626
Windows	loglite	93,242,803	344.411	177.369
Windows	gzip6	1,670,558,625	201.814	98.737
Windows	xz6	128,613,372	631.764	96.526
Windows	zstd3	191,939,206	201.003	87.376
Thunderbird	semzip	491,672,374	1493.123	466.335
Thunderbird	delog	469,712,868	1128.875	431.954
Thunderbird	loglite	799,163,025	783.912	300.608
Thunderbird	gzip6	2,017,676,108	264.485	141.320
Thunderbird	xz6	1,113,970,476	1768.539	164.796
Thunderbird	zstd3	1,688,621,662	250.181	132.701

1.3.5 Every saved rule and its observed activity

Table 10. Every saved rule in successful complete-file deployments, including inactive rules. These are observed admitted matches and stored values, not causal savings.

Dataset	Tag	Static route	Admitted matches	Stored values
Linux	TS2	Python inverse	2,257	2,257
Linux	TS	Python inverse	26,727	26,727
Linux	IP	Generic codec	6,080	6,080
Linux	H	Python checked/generic	10,404	10,404
Linux	E	Python checked/generic	1,179	1,179
Linux	B	Python checked/generic	4,996	4,996
Linux	C	Python checked/generic	984	984

Table 10. Every saved rule in successful complete-file deployments, including inactive rules. These are observed admitted matches and stored values, not causal savings.

Dataset	Tag	Static route	Admitted matches	Stored values
Linux	I	Python inverse	113	113
Proxifier	DU	Python inverse	7,587	7,587
Proxifier	HP	Python inverse	21,329	21,329
Proxifier	BS	Python inverse	10,148	10,148
Proxifier	BR	Python inverse	9,887	9,887
Proxifier	F	Python checked/generic	32,123	32,123
Proxifier	B	Python checked/generic	21,312	21,312
Apache	TS0	Python inverse	52,004	52,004
Apache	IP	Generic codec	31,116	31,116
Apache	G	Python checked/generic	3,301	3,301
Zookeeper	TS0	Python inverse	74,380	74,380
Zookeeper	DU	Python inverse	1,322	1,322
Zookeeper	HP	Generic codec	68,303	68,303
Zookeeper	E	Python checked/generic	3,856	3,856
Zookeeper	G	Python checked/generic	1,749	1,749
Zookeeper	B	Python checked/generic	53,065	53,065
Zookeeper	I	Python checked/generic	0	0
Mac	TS1	Python checked/generic	658	658
Mac	TS	Python inverse	107,145	107,145
Mac	PD	Explicit schema	100,162	100,162
Mac	K	Python checked/generic	3,930	3,930
Mac	G	Python checked/generic	1,770	1,770
Mac	F	Python checked/generic	925	925
Mac	C	Python checked/generic	1,118	1,118
Mac	RV0	Explicit schema	0	0
HealthApp	TS	Python inverse	114,535	114,535
HPC	TS	Python inverse	775,278	775,278
HPC	D	Python checked/generic	212,358	212,358
HPC	B	Python checked/generic	5,832	5,832
HPC	E	Python checked/generic	0	0
Hadoop	TS0	Python inverse	180,895	180,895
Hadoop	PT	Generic codec	1,741	1,741
Hadoop	H	Python inverse	25,378	25,378
Hadoop	B	Python checked/generic	83,601	83,601
Hadoop	D	Python checked/generic	4,579	4,579
Hadoop	E	Python checked/generic	0	0
Hadoop	J	Python checked/generic	0	0
Hadoop	F	Python checked/generic	1,804	1,804
Hadoop	G	Python checked/generic	0	0
OpenStack	TS1	Python inverse	207,820	207,820
OpenStack	TS	Python checked/generic	92,963	92,963
OpenStack	C	Python inverse	98,733	98,733
OpenStack	D	Python inverse	98,733	98,733
OpenStack	E	Python checked/generic	51,553	51,553
OpenStack	I	Python checked/generic	0	0
OpenStack	M	Python checked/generic	0	0
OpenStack	G	Python checked/generic	0	0
OpenStack	H	Python checked/generic	192,174	192,174
OpenStack	K	Python checked/generic	0	0
OpenStack	L	Python checked/generic	0	0
OpenStack	N	Python checked/generic	0	0
OpenStack	J	Python checked/generic	0	0
OpenStack	P	Python checked/generic	0	0

Table 10. Every saved rule in successful complete-file deployments, including inactive rules. These are observed admitted matches and stored values, not causal savings.

Dataset	Tag	Static route	Admitted matches	Stored values
OpenStack	Q	Python checked/generic	2,064	2,064
OpenStack	R	Python checked/generic	0	0
OpenStack	S	Python checked/generic	2,064	2,064
OpenStack	RV0	Explicit schema	0	0
OpenStack	RV1	Explicit schema	0	0
OpenSSH	TS0	Python inverse	655,147	655,147
OpenSSH	IP	Generic codec	542,851	542,851
OpenSSH	PD	Python inverse	655,147	655,147
OpenSSH	PT	Python inverse	198,235	198,235
OpenSSH	US	Python inverse	71,185	71,185
OpenSSH	G	Python checked/generic	254,867	254,867
Android	TS1	Python inverse	1,555,005	1,555,005
Android	B	Python checked/generic	4,225	4,225
Android	G	Python checked/generic	19,997	19,997
Android	D	Python checked/generic	2,242	2,242
Android	E	Python checked/generic	2,695	2,695
BGL	TS1	Python inverse	3,987,801	3,987,801
BGL	F	Python checked/generic	3,225,003	3,225,003
BGL	B	Python checked/generic	0	0
BGL	A	Python checked/generic	0	0
BGL	C	Python checked/generic	0	0
BGL	G	Python checked/generic	0	0
BGL	D	Python checked/generic	0	0
HDFS	TS	Python inverse	0	0
HDFS	D	Python checked/generic	12,577,685	12,577,685
HDFS	E	Python checked/generic	0	0
Spark	TS	Python inverse	27,410,250	27,410,250
Spark	T	Python inverse	11,069,157	11,069,157
Spark	C	Python checked/generic	2,421,815	2,421,815
Spark	U	Python inverse	1,033,632	1,033,632
Spark	F	Python inverse	0	0
Spark	G	Python inverse	516,633	516,633
Spark	V	Python inverse	633,548	633,548
Spark	W	Python inverse	5,112	5,112
Spark	R	Python checked/generic	24,670	24,670
Spark	X	Python checked/generic	3,393,976	3,393,976
Spark	K	Python inverse	265,248	265,248
Spark	L	Python inverse	12,950	12,950
Spark	M	Python checked/generic	725,780	725,780
Spark	H	Python checked/generic	744,891	744,891
Spark	I	Python checked/generic	0	0
Spark	O	Python checked/generic	1,060	1,060
Spark	P	Python checked/generic	1,024	1,024
Spark	Y	Python inverse	749	749
Windows	TS0	Python inverse	114,422,286	114,422,286
Windows	B	Python inverse	43,953,830	43,953,830
Windows	C	Python inverse	43,953,830	43,953,830
Windows	D	Python inverse	43,953,830	43,953,830
Windows	R	Python checked/generic	437,824	437,824
Windows	K	Python checked/generic	9,713,087	9,713,087
Windows	L	Python checked/generic	0	0
Windows	F	Python checked/generic	10,626,040	10,626,040
Windows	G	Python checked/generic	7,302,165	7,302,165

Table 10. Every saved rule in successful complete-file deployments, including inactive rules. These are observed admitted matches and stored values, not causal savings.

Dataset	Tag	Static route	Admitted matches	Stored values
Windows	O	Python checked/generic	101,747	101,747
Windows	S	Python checked/generic	2,371,534	2,371,534
Windows	P	Python checked/generic	0	0
Windows	M	Python checked/generic	1,270,186	1,270,186
Windows	H	Python checked/generic	0	0
Windows	I	Python checked/generic	1,502,285	1,502,285
Windows	N	Python checked/generic	0	0
Thunderbird	RV0	Explicit schema	0	0
Thunderbird	RV1	Explicit schema	0	0

1.3.6 Original construction costs and corrected fitting

Table 11. Original construction publication and corrected fitting costs in seconds. The original publication includes synthesis, prior sampling/startup, and the old fit, including failures. The synthesis subprocess is contained in that publication time, so columns must not be added as independent costs. Corrected fitting is a separate zero-API replay.

Dataset	Original fit	Original publication	Synthesis component	Corrected fit
Linux	PASS	103.778	96.642	5.286
Proxifier	PASS	34.918	27.380	6.167
Apache	PASS	105.661	97.541	5.308
Zookeeper	PASS	151.458	131.185	16.904
Mac	FAILED	184.475	160.984	18.784
HealthApp	PASS	41.013	32.292	5.129
HPC	PASS	84.327	69.443	11.102
Hadoop	PASS	139.297	113.184	19.913
OpenStack	PASS	143.857	99.341	33.262
OpenSSH	PASS	97.445	72.310	21.617
Android	PASS	99.656	87.082	8.458
BGL	PASS	98.353	78.974	13.806
HDFS	PASS	42.773	33.150	4.448
Spark	PASS	168.977	141.715	17.541
Windows	PASS	160.756	133.038	16.914
Thunderbird	PASS	51.088	44.706	0.879

1.3.7 Descriptive semantic-guard task costs

Table 12. Descriptive first-pass sums for final successful semantic attempts. Inverse-check timing already includes cold-inverse state clearing, which also appears in the boundary-clear column, so the columns overlap. Failed-attempt details remain in guard logs and are not fully represented by these sums. Worker tasks overlap, so these are neither elapsed file latency nor subtractable overhead. Formal encoding includes all attempts, checks, and state clearing.

Dataset	Inverse-check task seconds	Boundary-clear task seconds	Recovered blocks
Linux	0.762	0.145	0
Proxifier	0.873	0.133	0
Apache	0.510	0.097	0
Zookeeper	2.113	0.107	0
Mac	2.956	0.189	0
HealthApp	1.294	0.361	0
HPC	4.000	0.545	0
Hadoop	4.542	0.794	0
OpenStack	6.084	0.384	0
OpenSSH	14.409	0.557	0
Android	11.553	0.609	0
BGL	39.007	1.177	0
HDFS	41.550	1.326	0
Spark	576.773	2.473	0
Windows	1482.664	5.969	0
Thunderbird	247.526	8.516	0

2 Repeated syntheses and cost-guided selection

Every synthesis uses the unchanged trainer, sampler, prompts, forty-call ceiling, zero HTTP retries, and GPT-4o through the same gateway. Only the temperature and output directory differ from the greedy run. All selection evaluations fit a storage policy on original block 0, encode it, decode the archive in a separate process, and compare SHA-256 digests. The held-out sample of each multi-block file contains up to eight evenly spaced original blocks from block 1 onward and is never read by synthesis, gating, pooling, or fitting. The repeated greedy synthesis (g1) is reported for reproducibility and never enters selection.

Table 15. All syntheses per dataset (B0 is the selection-time evaluation, including the manifest, while the formal deployed block-0 payload can differ slightly, e.g. Thunderbird 283,484 bytes without the manifest). Rules counts verified rule groups before/after the quality gate. B0 gives complete block-0 archive bytes (single/gated). HO gives held-out sample archive bytes (single/gated), with – for one-block files. c0 is the greedy synthesis (SemZip-1), c1–c4 are temperature-0.7 samples, and g1 is a repeated greedy synthesis that is never pooled or gated.

Dataset	Synth.	Calls	Rules	B0 single	B0 gated	HO single	HO gated
Linux	c0	17	8/4	61,085	60,325	–	–
	c1	23	5/4	61,321	61,265	–	–
	c2	14	7/7	60,121	60,121	–	–
	c3	13	7/7	58,613	58,613	–	–
	c4	14	8/5	59,993	58,805	–	–
	g1	14	–	–	–	–	–
	pool	–	7	start c3	58,613	–	–
Proxifier	c0	3	6/5	73,553	73,497	–	–
	c1	7	7/5	85,141	84,685	–	–

Table 15. (continued)

Dataset	Synth.	Calls	Rules	B0 single	B0 gated	HO single	HO gated
	c2	6	8/6	73,537	73,109	—	—
	c3	4	5/5	76,509	76,509	—	—
	c4	4	5/5	76,189	76,189	—	—
	g1	4	—	—	—	—	—
	pool	—	7	start c2	70,361	—	—
Apache	c0	16	3/2	71,282	71,110	—	—
	c1	10	6/5	69,906	69,710	—	—
	c2	11	7/6	70,010	69,834	—	—
	c3	10	6/5	69,910	69,766	—	—
	c4	16	3/2	71,154	71,046	—	—
	g1	7	—	—	—	—	—
	pool	—	5	start c1	69,646	—	—
Zookeeper	c0	17	7/5	58,249	56,745	—	—
	c1	9	10/5	70,173	65,929	—	—
	c2	9	9/6	58,301	57,917	—	—
	c3	6	7/7	60,065	60,065	—	—
	c4	5	7/5	90,449	64,601	—	—
	g1	18	—	—	—	—	—
	pool	—	4	start c0	55,045	—	—
Mac	c0	28	8/6	266,055	241,067	78,323	74,239
	c1	22	9/6	293,343	245,011	85,911	73,735
	c2	28	11/8	277,155	248,855	79,091	74,455
	c3	28	9/6	277,879	245,903	79,131	74,111
	c4	13	6/4	246,259	243,387	75,751	74,667
	g1	23	—	—	—	78,331	—
	pool	—	12	start c0	237,723	—	74,251
HealthApp	c0	4	1/1	230,145	148,053	381,375	247,347
	c1	4	3/3	333,745	316,517	529,523	493,131
	c2	11	0/0	216,981	216,981	373,655	373,655
	c3	5	0/0	216,981	216,981	373,655	373,655
	c4	2	1/1	230,305	148,089	381,307	247,759
	g1	4	—	—	—	381,359	—
	pool	—	1	start c0	148,053	—	247,347
HPC	c0	16	4/2	260,227	192,651	800,949	543,277
	c1	9	1/1	195,087	195,087	554,141	554,141
	c2	23	6/4	271,247	194,235	813,321	555,185
	c3	9	7/4	196,159	190,827	572,961	542,449
	c4	14	8/6	194,887	194,351	547,057	546,993
	g1	29	—	—	—	554,461	—
	pool	—	5	start c3	190,395	—	542,569
Hadoop	c0	17	9/7	286,602	286,442	288,430	288,794
	c1	14	4/3	380,518	380,150	347,870	347,598
	c2	17	8/6	381,086	380,118	349,046	347,458
	c3	15	14/10	325,846	307,610	312,554	311,198
	c4	14	12/10	367,714	366,222	335,934	334,898
	g1	12	—	—	—	291,414	—
	pool	—	10	start c0	284,130	—	287,494
OpenStack	c0	11	19/19	1,255,921	1,255,921	1,191,623	1,191,623
	c1	12	6/5	1,344,853	1,338,969	1,265,835	1,264,775
	c2	14	10/7	1,325,557	1,299,513	1,252,471	1,230,887
	c3	13	7/6	1,244,609	1,242,117	1,176,395	1,220,987
	c4	12	15/11	1,340,521	1,335,261	1,267,319	1,266,191

Table 15. (continued)

Dataset	Synth.	Calls	Rules	B0 single	B0 gated	HO single	HO gated
	g1	9	–	–	–	1,255,587	–
	pool	–	7	start c3	1,223,421	–	1,210,963
OpenSSH	c0	6	6/5	77,255	76,183	559,397	547,973
	c1	12	5/5	82,303	82,303	592,921	592,921
	c2	9	6/5	77,431	76,211	555,693	548,001
	c3	13	6/4	81,291	80,311	580,905	575,237
	c4	6	6/4	79,767	78,039	569,581	561,021
	g1	8	–	–	–	559,273	–
	pool	–	5	start c0	76,063	–	547,385
Android	c0	9	5/4	376,547	376,499	2,629,773	2,629,961
	c1	13	1/1	381,955	381,955	2,629,661	2,629,661
	c2	15	2/2	382,007	382,007	2,631,249	2,631,249
	c3	13	2/2	377,327	377,327	2,629,509	2,629,509
	c4	6	2/2	394,475	394,475	2,755,201	2,755,201
	g1	12	–	–	–	2,629,549	–
	pool	–	4	start c0	376,499	–	2,629,961
BGL	c0	8	7/6	377,471	377,191	2,832,285	2,832,149
	c1	14	7/5	392,203	331,079	2,859,697	2,559,481
	c2	8	5/5	318,403	317,687	2,543,137	2,542,789
	c3	9	4/3	319,119	312,915	2,588,649	2,535,305
	c4	9	9/8	378,151	377,723	2,921,013	2,921,233
	g1	7	–	–	–	2,974,597	–
	pool	–	6	start c3	272,715	–	2,122,873
HDFS	c0	6	3/3	440,072	440,072	4,100,430	4,100,430
	c1	9	5/5	376,832	376,500	3,688,974	3,592,850
	c2	10	4/4	376,500	376,500	3,593,322	3,593,322
	c3	10	5/4	376,500	376,088	3,593,322	3,592,446
	c4	6	2/2	467,700	467,700	4,324,206	4,324,206
	g1	14	–	–	–	3,586,158	–
	pool	–	4	start c3	376,088	–	3,592,446
Spark	c0	18	18/14	240,845	236,421	1,260,383	1,147,083
	c1	10	7/4	254,581	242,417	1,169,563	1,167,203
	c2	7	7/3	253,673	245,073	1,227,511	1,170,859
	c3	25	5/2	253,021	244,197	1,170,263	1,169,983
	c4	8	6/3	247,769	244,453	1,179,951	1,175,163
	g1	14	–	–	–	1,170,639	–
	pool	–	15	start c0	233,089	–	1,142,343
Windows	c0	22	16/13	46,875	46,183	296,797	290,145
	c1	28	14/9	46,683	45,327	298,897	291,093
	c2	18	9/5	49,275	47,895	328,133	315,541
	c3	29	17/12	44,739	44,119	267,317	258,237
	c4	21	11/8	47,979	47,091	322,313	304,557
	g1	25	–	–	–	307,429	–
	pool	–	13	start c3	43,387	–	252,661
Thunderbird	c0	4	2/2	293,195	293,195	2,131,225	2,131,225
	c1	15	6/5	293,459	293,107	2,135,017	2,133,985
	c2	4	7/4	321,223	291,915	2,366,673	2,133,973
	c3	17	12/9	288,971	283,643	2,156,221	2,127,177
	c4	7	6/4	293,519	292,695	2,134,213	2,131,345
	g1	12	–	–	–	2,059,833	–
	pool	–	9	start c3	283,643	–	2,127,177

Table 13. Archive size of each single synthesis relative to the selected deployment (100 = cost-guided selection, lower is better). Multi-block files use the fixed held-out block sample, while † one-block files use block 0 (in-sample). *c0* is the greedy synthesis deployed as SemZip-1, and *c1*–*c4* are temperature-0.7 samples. Single uses verified programs as synthesized. Gated is measured after the per-synthesis quality gate.

Dataset	c0 single	c0 gated	single range (c0–c4)	gated range
Linux†	104.2	102.9	100.0–104.6	100.0–104.5
Proxifier†	104.5	104.5	104.5–121.0	103.9–120.4
Apache†	102.3	102.1	100.4–102.3	100.1–102.1
Zookeeper†	105.8	103.1	105.8–164.3	103.1–119.8
Mac	105.5	100.0	102.0–115.7	99.3–100.6
HealthApp	154.2	100.0	151.1–214.1	100.0–199.4
HPC	147.6	100.1	100.8–149.9	100.0–102.3
Hadoop	100.3	100.5	100.3–121.4	100.5–120.9
OpenStack	98.4	98.4	97.1–104.7	98.4–104.6
OpenSSH	102.2	100.1	101.5–108.3	100.1–108.3
Android	100.0	100.0	100.0–104.8	100.0–104.8
BGL	133.4	133.4	119.8–137.6	119.4–137.6
HDFS	114.1	114.1	100.0–120.4	100.0–120.4
Spark	110.3	100.4	102.4–110.3	100.4–102.9
Windows	117.5	114.8	105.8–129.9	102.2–124.9
Thunderbird	100.2	100.2	100.2–111.3	100.0–100.3

3 Guard regressions and historical storage control

Zero-API regression replay exercises two previously observed failures, namely a held-out literal-placeholder collision and a layout-dependent inverse paired with a layout-omitting codec. Under the guard, each takes the fixed residual-only recovery and passes independent native-plus-semantic archive decoding. A previously valid training block keeps its original semantic archive bytes. These regression results do not change the historical backbone failure counts. Propagated infrastructure failures and failed recovery still abort. Additional controls cover mutable encoder memo state and block-order independence. The historical saved-plan control averages $93.82\times$ with fixed storage and $94.61\times$ with adaptive storage, and fixed storage adds 0.626% total bytes (Section 9).

4 Uniform Deployment Configuration

5 First-block Training Outcomes and Costs

Table 17. One first-block training per dataset, followed by uniform corrected storage fitting. All sixteen syntheses and corrected fits pass. Tokens are observed provider totals, and time is wall seconds. Synthesis measures its subprocess and excludes preceding scanning/sampling. Fit includes its process startup. Earlier fitting failures remain in the artifact.

Dataset	Samples	Rules	Calls	Prompt tokens	Output tokens	Synthesis (s)	Fit (s)
Linux	80	8	17	74,426	13,042	96.6	5.3
Proxifier	80	6	3	27,131	4,794	27.4	6.2
Apache	79	3	16	66,209	16,018	97.5	5.3
Zookeeper	54	7	17	53,508	22,971	131.2	16.9
Mac	79	8	28	78,421	31,235	161.0	18.8
HealthApp	80	1	4	20,821	6,775	32.3	5.1
HPC	80	4	16	60,820	10,206	69.4	11.1
Hadoop	80	9	17	64,266	19,602	113.2	19.9
OpenStack	80	19	11	113,551	19,982	99.3	33.3
OpenSSH	78	6	6	58,760	13,764	72.3	21.6
Android	80	5	9	51,650	18,175	87.1	8.5
BGL	76	7	8	60,770	15,353	79.0	13.8

Table 17. (continued)

Dataset	Samples	Rules	Calls	Prompt tokens	Output tokens	Synthesis (s)	Fit (s)
HDFS	26	3	6	25,588	4,450	33.1	4.4
Spark	80	18	18	59,767	28,466	141.7	17.5
Windows	80	16	22	110,868	20,816	133.0	16.9
Thunderbird	80	2	4	56,283	9,517	44.7	0.9
Total	1192	122	202	982,839	255,166	1419.0	205.5

The 202 requests consume 1,238,005 observed tokens. Recorded synthesis subprocesses plus corrected fitting total 27.07 minutes. This excludes preceding scanning/sampling and earlier superseded fitting attempts, and is not a total offline-latency measure. No additional model call is used for policy correction or block-state correction.

Table 18. Separate zero-API sampling replay in seconds. Every sample is byte-identical to its original. The writer includes its own second scan. Source hashing and imports are outside these timers.

Dataset	Count scan	Sample writer	Total
Linux	0.003	0.928	0.932
Proxifier	0.003	0.893	0.897
Apache	0.007	1.875	1.882
Zookeeper	0.010	3.084	3.094
Mac	0.016	4.695	4.711
HealthApp	0.010	2.652	2.662
HPC	0.011	2.905	2.916
Hadoop	0.020	5.112	5.132
OpenStack	0.021	10.278	10.298
OpenSSH	0.013	3.835	3.847
Android	0.014	4.599	4.614
BGL	0.011	4.572	4.583
HDFS	0.012	4.244	4.256
Spark	0.013	4.030	4.043
Windows	0.018	6.535	6.554
Thunderbird	0.015	5.282	5.297
Total	0.198	65.520	65.718

This additional replay does not add API requests or training outcomes. Its duration is not backfilled into the original synthesis timer or added to it as an observed end-to-end training time. The host was shared, input hashes had already been read, and caches were not dropped.

5.1 Saved-response replay and training environment

A zero-API control executes the original trainer on each saved sample in a fresh local Python 3.9.6 process, with UTC0 and the C locale. Its hook requires an exact original prompt, parses the corresponding saved response with the original parser, and stops immediately after the original plan writer emits the plan. The trainer’s sample-level auxiliary files are written, but no complete archive, storage fit, or replacement deployment is produced. Fourteen of sixteen parsed plans and plan-file hashes match exactly. Linux consumes 13 of 17 saved responses and Mac 4 of 28. Both plans differ. An unmatched prompt raises without obtaining a new response. The original compiler may catch a repair error, so unmatched records are explicitly retained.

All 28 unmatched repair prompts contain year-1900 timestamp counterexamples. Their saved microseconds differ from UTC0 by a constant historical local-mean-time offset of a non-UTC zone. Two separate explanatory replays change only the control timezone to that non-UTC zone. Linux consumes 17/17 and Mac 28/28 saved responses, reproducing both original plan objects and file hashes. The original launcher inherits its process timezone without recording or pinning it. The numeric correspondence is evidence about these counterexamples, not a recovered original environment snapshot.

Table 14. Offline cost of the four additional syntheses and of selection, per dataset. Requests and synthesis time come from the retained training records. Evaluations are complete block-0 archive-cost evaluations (fit, encode, separate decode, SHA-256), with summed wall time on the shared host.

Dataset	Requests	Synthesis (s)	Gate evals	Gate (s)	Pool evals	Pool (s)
Linux	64	371	98	2325	26	934
Proxifier	21	166	74	1530	76	2565
Apache	47	325	59	1104	51	2367
Zookeeper	29	331	132	7415	132	14622
Mac	91	569	148	11834	170	13256
HealthApp	22	158	14	532	8	1234
HPC	55	302	80	2718	67	2000
Hadoop	60	436	157	19616	118	11626
OpenStack	51	391	156	19000	192	27605
OpenSSH	40	321	67	6590	54	8270
Android	47	314	29	557	18	604
BGL	40	303	73	2230	175	33944
HDFS	35	165	40	587	37	844
Spark	50	401	178	5532	49	4328
Windows	96	497	312	13455	208	18967
Thunderbird	43	341	104	2126	26	710

The original UTC0 mismatches remain in the denominator. These checks validate the saved-response/compiler boundary under stated environments, not HTTP transport, provider identity, or the entire original sampling/network/fitting pipeline. All source files, samples, original exchanges, and deployed plans remain unchanged.

6 Historical SemZip-1 matched-representation control (four datasets, superseded by Table 29)

Table 19. Representation archive components in bytes. Semantic payloads include their actual compressed metadata and program descriptions. Raw member sizes are not added again.

Dataset	Branch	Semantic	Residual	Manifest
Linux	latent	30,752	30,180	153
Linux	surface	36,372	30,180	153
HPC	latent	868,492	192,292	271
HPC	surface	858,840	192,292	271
OpenSSH	latent	575,140	61,052	335
OpenSSH	surface	689,916	61,052	335
Android	latent	525,516	5,039,724	606
Android	surface	564,200	5,039,724	606

6.1 Descriptive negative cases

These two completed external pairs are inspected after observing their larger archives. This is descriptive accounting, not a newly selected ablation cohort. SemZip-1’s HPC and HealthApp archives were respectively 41.36% and 33.56% larger than official DeLog’s, with exact reconstruction and zero recovered blocks. For HPC, three of four saved rules are active and semantic payloads constitute 81.85% of the archive. Its latent branch is 0.92% larger than the matched surface control, which remains 40.07% larger than DeLog. HealthApp’s only timestamp rule covers 45.20% of LF records and its residual payload accounts for 86.28% of the archive. Outer manifests account for less than 0.04% in either dataset. These observations locate byte costs without isolating the cause of the external gap. Matched replay measures normalization together with coding, the official recognizers and derived residual processing differ, and this historical cohort lacks a HealthApp matched-representation control.

Table 16. Uniform configuration of the current first-block-trained method. Every result must identify its exact policy-export and guard revision, while historical cohorts retain their original settings.

Component	Fixed contract
Training input	Original block 0 only. At most 100,000 physical LF records, unchanged tail. Five retained syntheses per dataset (one greedy, four at temperature 0.7). A repeated greedy synthesis is diagnostic only.
Sampler	Generic skeletons with $h = 8$, at most two examples/group, support 50 and top 40 groups. No dataset-named rule selection.
Default service	GPT-4o at temperature 0 (greedy) and 0.7 (four samples). Requested output ceiling 16,384 tokens, at most 40 calls, no HTTP retries.
Program contract	Matcher plus executable forward/inverse. Composed numeric, formatted numeric, or exact common-variable output.
Deterministic stages	Capture/route checks, bounded reconstruction-guided repair, width/layout-preserving lowering, ordered replay, and type-compatible physical codecs. Per-plan categories are audited separately.
Selection	Per-synthesis gate and cost-guided pooling on block 0 (Algorithm 1 of the main paper, 4 rounds, 600 evaluations).
Storage fitting	Block-0 staged archive-cost selection over identical-file aliases, whole-value/shared dictionaries, columns, numeric coding. Exported policy must reproduce the selected training archive cost. Later blocks perform no candidate search.
Online execution	Four block workers, 100,000 original LF records/block. All dictionaries, differences, aliases and data caches reset per block. No LLM calls outside the optional evolution stage.
Recovery	Actual semantic-archive inverse check. One fixed empty-program/policy recovery for recognized semantic contract errors, otherwise explicit failure. Attempt and recovery costs count online.
Backend	Semantic tar/XZ uses Python LZMA preset 6 extreme. Residual uses the DeLog-derived native libarchive XZ path. Complete decoder-required metadata counted.

7 Archive-boundary and version controls

7.1 Formal measurement audit correction

The first formal controller is retained as an aborted instrumentation campaign with 29 passed observations, two audit failures, one interrupted in-flight observation, and 184 not-started observations. Its controller groups the 185 unfinished rows together. The interrupted trial retains its original running record and is identified separately in the evidence archive. Two post-decode inventory checks report a file-identity change, one for SemZip/Linux and one for LogLite/Proxifier, after both codec phases exit successfully. Subsequent independent full-file and per-block inventories of both retained outputs match the original inputs. A separate probe of newly written files reproduces a five-microsecond modification-time adjustment on the shared filesystem without running any codec. These are metadata-audit failures, not established byte-reconstruction mismatches.

The corrected controller still independently compares complete and per-block hashes and retains strict size, device, inode, and path checks. It records a newly closed output’s modification-time change without rejecting an otherwise exact reconstruction, while original-input identity remains strict. The audit stays outside unchanged outer-process encoding and decoding timers. All six methods receive a fresh complete 216-trial schedule. No observation from the superseded campaign is pooled into it or selectively substituted. New-file, retained-output, corruption, and explicit metadata-branch controls are kept separately from the measured trials. All 216 corrected-controller trials pass. Each three-repeat archive set is identical to its accepted first pass, and the final archive and source/input-lock checks pass. No corrected-controller trial observes an output modification-time change.

7.2 Archive-directory reconstruction and fixed-policy controls

The external wrapper uses fixed numbered archive filenames. Its formal decoder reads a reconstructible index from the encoding ledger, using only the archive path and block index. It does not use original lengths, hashes, records, or input paths to decode. A separate directory-only control reconstructs that index from copied archive filenames and gives its decoder no original input, encoding ledger, expected hash, length, or tail state. All eight fixtures pass independent materialized-file checks, covering five methods on the tiny fixture plus gzip, XZ, and Zstd on 100,001-record two-block inputs without a final LF. These eleven existing blocks are decoded without recompression. This establishes

Table 20. Zero-API regression replay under the cold semantic archive guard. The scope of each restoration check is explicit. Historical failures remain failures in their original cohorts.

Saved-plan/input case	Recovery	Verified result
Held-out literal-placeholder collision	Fixed residual-only	Full decode passes
Layout-dependent inverse/codec mismatch	Fixed residual-only	Full decode passes
Valid original training block	None, same semantic bytes	Full decode passes
Inverse depending on encoder memo state	Fixed residual-only	Semantic inverse passes, native not run
Two blocks in different orders (five executions)	None, same payloads	Full decode passes

the tested directory-decoding scope but does not change the frozen formal wrapper or its timer boundaries. Fixed filename conventions and filesystem directory metadata are treated consistently as the container interface. LogLite’s data-dependent final-LF flag is one counted archive byte per block.

The first storage exporter omitted some logical subfield codec decisions after file aliasing removed duplicate file keys. Mac’s original strict training-block replay therefore differed by 44 semantic archive bytes and was retained as a failed fit. Exporting those decisions before aliasing restores its original 57,896-byte semantic archive. The same corrected exporter is applied to all sixteen original programs and training blocks, with sixteen passing fixed-policy replays and no new model call. No suffix result selects a policy. The original failed fit and its cost remain in the evidence.

The cold inverse guard additionally clears input-derived caches and potentially stateful compiled callables before decoding the written semantic payload, as well as at block boundaries. A mutable encoder-memo fixture distinguishes the prior warm acceptance from the corrected cold inverse and its fixed recovery. That fixture tests semantic reconstruction only. Separate native-plus-semantic regression checks and block-order replays retain their own full-decoding scope.

7.3 LogLite compatibility preflights

The official byte-stream decoder eagerly reserves 33 GiB, which fails an 8-GiB address-space-limited tiny-input probe. The disclosed resource adaptation changes that reservation to zero. The evaluated LogLite snapshot is commit 68f851e, and its output is compressed with XZ 6. A fourteen-case synthetic preflight of the resource-only profile retains three failures, namely an embedded-NUL change, a long equal-length-record pair overflowing the run header, and an original-record-length overflow. The uniform wide profile adopts the upstream-recommended 16-bit run header and 23-bit original-length field, together with the reserve change and counted final-LF flag. It passes thirteen cases. The synthetic embedded-NUL mismatch remains unrepaired. No dataset-specific tuning or NUL-specific fallback is added.

A census of all sixteen unchanged input files finds NUL bytes only in HealthApp block 2 (106) and BGL block 42 (4). Both complete original blocks reconstruct exactly with the frozen wide profile. Those passes do not repair the synthetic failure or establish success on an unmeasured file. Every admitted complete-file observation still requires its own independent restoration. The artifact retains the source patch, executable identities, original failed fixtures, and successful actual-byte controls.

8 Source-study evidence and scope

The source census records 2,781 repositories, 285,134 logging calls, 24,108 candidate rows, 15,659 strict detections, and 1,520 repositories with strict detections. Its preserved holdout contains 449 detector-positive calls and 50 ordinary-call probes, with 499 explicit row-level decisions bound to the frozen sample and detector hashes. Recorded labels agree with all sampled detector labels, and their per-class 95% Wilson lower bounds range from 92.9% to 95.3%. Four additional labels are recorded as missed, including one of the fifty ordinary-call probes. These are agreement statistics conditional on the saved labels. Reviewer provenance is not independently established, and finite agreement does not prove zero census-wide false positives or full recall. The anonymous package retains the candidate ledger, sample, decisions, source identities, and metadata verifier.

The paired-field diagnostic retains all four original inputs, twenty-three source-linked families, exact source revisions and rendering locations, and recorded archive hashes. Coverage counts unframed rendered-span bytes, while contraction divides length-framed surface-stream bytes by latent-stream bytes. For example, Spark uses 530,143,198 framed surface bytes and 37,786,126 latent bytes, yielding 14.030 $\times$. Its unframed rendered spans occupy 497,902,648 bytes. The latent

Table 21. Historical saved-plan complete-file storage control, not the new first-block-trained main cohort. Both methods use identical extraction plans, native binaries, input bytes and original 100k-line blocks. Positive archive change means fixed storage is larger. All rows pass independent full-file SHA-256.

Dataset	Raw MB	Blocks	Adaptive	Fixed	Archive change
Android	192.27	16	33.01×	33.01×	+0.00000%
Apache	5.14	1	73.36×	73.36×	+0.00000%
BGL	743.19	48	46.50×	45.99×	+1.11951%
HDFS	1,577.98	112	27.84×	27.01×	+3.05877%
HPC	33.55	5	44.74×	44.74×	+0.00000%
Hadoop	48.60	4	80.49×	80.36×	+0.15768%
HealthApp	23.53	3	59.52×	59.52×	+0.00000%
Linux	2.35	1	39.32×	39.32×	+0.00000%
Mac	16.88	2	50.61×	50.61×	+0.00000%
OpenSSH	73.42	7	116.29×	116.29×	+0.00000%
OpenStack	61.44	3	23.53×	23.53×	+0.00046%
Proxifier	2.54	1	33.09×	33.09×	+0.00000%
Spark	2,941.22	333	62.71×	62.50×	+0.33644%
Thunderbird	31,788.30	2113	66.16×	66.00×	+0.23934%
Windows	28,012.70	1147	584.57×	573.72×	+1.89209%
Zookeeper	10.43	1	172.08×	172.08×	+0.00000%
Arithmetic mean	—	—	94.61×	93.82×	—

stream includes difference and variable-length integer coding with whole-file per-family state. This historical diagnostic is separate from the strictly independent 100,000-record deployment and its matched-representation experiment. Its Spark source assembly differs from the main benchmark by one additional LF after each of the 3,852 source members, including the last member. A separate local read-only control verifies the published source archive’s recorded identity, every member’s content hash, and both complete assembly hashes. It also independently hashes the existing historical file and checks every added-LF boundary. This establishes a byte-exact reconstruction recipe without identifying the original assembly script or changing either input. The package documents the public archive entry and recipe. It also documents the official full-size LogBench download entry. The current remote LogBench payload was not downloaded or rehashed in this packaging check. Other source-study packaging checks validate preserved counts, identities, and arithmetic but do not rerun field matching, compression, or the original study’s archive reconstruction.

9 Historical saved-plan storage control

This cohort changes storage selection while holding historical extraction programs fixed. Some programs may have used a longer synthesis prefix. Fitting a storage policy on block 0 does not turn these into first-block-only program trainings. All values below retain their original cohort and timer boundaries. All sixteen pairs reconstruct independently. Nine pairs have byte-identical archives, while fixed storage increases total archive size by 0.626%. The arithmetic means are 93.82× and 94.61×. They are not the new guarded-method mean or an external-baseline comparison. All seven archive regressions remain in the table. All individual timing observations appear in Section 21. The twelve files no larger than BGL have three alternating formal pairs after an initial pass. Encoding median changes range from -4.3% to +32.3%. Apache regresses, and ranges overlap on five files. These historical in-process timers exclude interpreter startup/imports. They cannot be compared directly with the new external outer-CLI timers. All 104 full-file and 8,146 block reconstructions pass, and repeated archives match their first-pass files. The complete BGL archive also decodes with original inputs and external plans unavailable.

10 Historical learned-layer removal control

Linux/HPC use historical programs, while OpenSSH/Android use the older fresh first-block pilot plans. Removing the learned layer also removes its field separation and storage, so the result does not isolate latent normalization. Linux, OpenSSH, and Android save 35.29%, 42.41%, and 7.71%. HPC increases by 0.12%. All eight complete files and 58 block reconstructions pass. This cohort is retained intact. The matched replay on the deployed plans (Table 29) supersedes it.

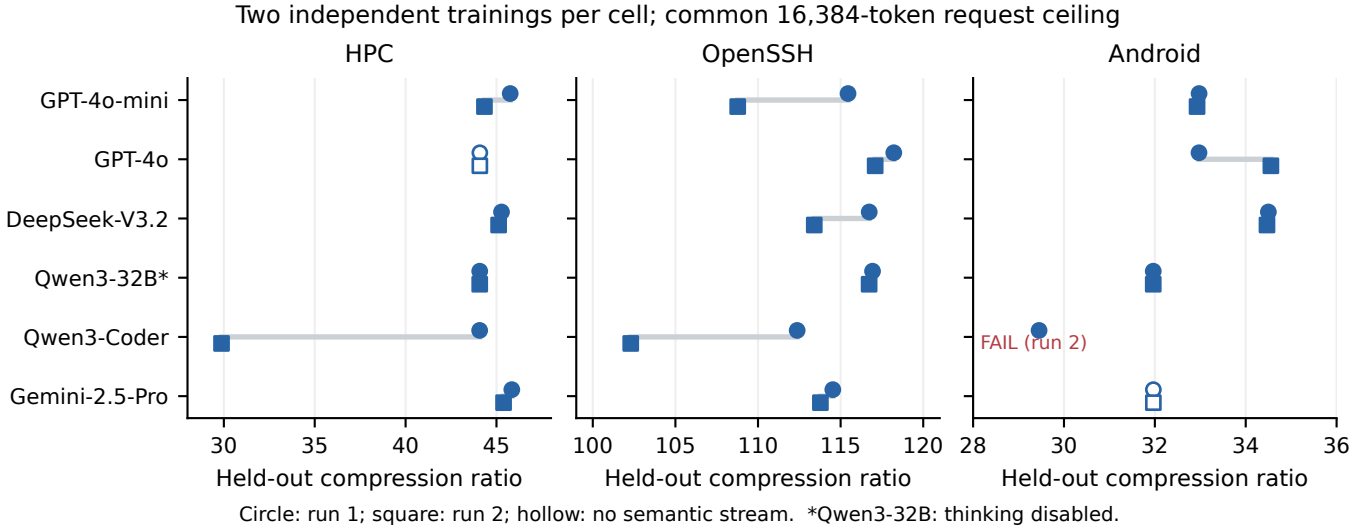
Table 22. Historical mixed-plan cohort. Learned-program layer vs. residual-only control on complete files. Both conditions share the native residual backend, and all rows pass full-file SHA-256. Positive reduction means a smaller archive.

Dataset	Residual only	Learned programs	Archive reduction
Linux	25.44×	39.32×	35.29%
HPC	44.79×	44.74×	-0.12%
OpenSSH	66.19×	114.94×	42.41%
Android	31.83×	34.48×	7.71%

Earlier proposal/repair replay. OpenSSH’s initial pilot uses five proposal and one LLM repair call, retaining eight rules in 69.61 seconds, while Android uses four proposal and thirteen repair calls, retaining two rules in 98.49 seconds. Replay of recorded model-response contents at the `call_llm` boundary reproduces both parsed plans. Prompts must match a saved request, and unmatched requests fail without an API call. Disabling only LLM repair produces the same parsed plans, with deterministic repairs retained. Thus fourteen repair calls have no marginal parsed-plan benefit in these two historical pilots. This does not replay the full HTTP stack or establish that repair is universally unnecessary.

11 Historical six-backbone comparison

Original block 0 trains the program and fixed storage policy, and original blocks 1–4 are held out. Blocks contain 100,000 physical LF records except unchanged original tails. Dictionaries, numeric state, and file aliases reset within every block. Two fresh API trainings per cell share the same sampled records and first proposer prompt by hash. All requests use temperature zero, at most forty calls, no HTTP retries, and a 16,384-token output ceiling.



Model	HPC	OpenSSH	Android	Decode pass
GPT-4o-mini	45.05	112.11	32.95	6/6
GPT-4o	44.09	117.66	33.76	6/6
DeepSeek-V3.2	45.20	115.07	34.48	6/6
Qwen3-32B	44.08	116.83	31.96	6/6
Qwen3-Coder	36.97	107.33	29.45*	5/6
Gemini-2.5-Pro	45.62	114.15	31.97	6/6

Ratios are arithmetic means of the two trainings within each dataset. *Android/Qwen3-Coder has only one valid ratio and one failed held-out reconstruction. It is not a two-run estimate. Both individual observations and every failure are retained below. Hollow points indicate no semantic streams, not failed decompression. A common requested ceiling does not equate provider tokenization, reasoning budget, or inference compute. Qwen3-32B disables thinking because its non-streaming endpoint requires it.

12 All 36 explicit-allowance outcomes

Dataset	Model	Run	Ratio	Entries	Active blocks	Calls	Synthesis s
HPC	GPT-4o-mini	1	45.756	12	4	11	173.9
HPC	GPT-4o-mini	2	44.339	6	4	6	386.5
HPC	GPT-4o	1	44.086	1	0	15	68.6
HPC	GPT-4o	2	44.086	1	0	12	63.5
HPC	DeepSeek-V3.2	1	45.277	3	4	7	149.9
HPC	DeepSeek-V3.2	2	45.118	18	4	11	176.9
HPC	Qwen3-32B	1	44.079	2	2	13	539.6
HPC	Qwen3-32B	2	44.079	2	2	13	590.2
HPC	Qwen3-Coder	1	44.075	2	2	12	209.1
HPC	Qwen3-Coder	2	29.872	32	4	17	449.8
HPC	Gemini-2.5-Pro	1	45.844	14	4	5	570.8
HPC	Gemini-2.5-Pro	2	45.390	13	4	5	631.0
OpenSSH	GPT-4o-mini	1	115.449	5	4	10	162.5
OpenSSH	GPT-4o-mini	2	108.772	5	4	10	198.0
OpenSSH	GPT-4o	1	118.219	6	4	7	68.1
OpenSSH	GPT-4o	2	117.091	7	4	10	82.5
OpenSSH	DeepSeek-V3.2	1	116.731	7	4	8	176.8
OpenSSH	DeepSeek-V3.2	2	113.405	7	4	9	250.1
OpenSSH	Qwen3-32B	1	116.931	6	4	10	382.1
OpenSSH	Qwen3-32B	2	116.725	7	4	12	409.3
OpenSSH	Qwen3-Coder	1	112.370	10	4	12	246.3
OpenSSH	Qwen3-Coder	2	102.297	8	4	18	281.1
OpenSSH	Gemini-2.5-Pro	1	114.528	4	4	6	651.8
OpenSSH	Gemini-2.5-Pro	2	113.776	5	4	8	725.1
Android	GPT-4o-mini	1	32.973	3	4	28	454.7
Android	GPT-4o-mini	2	32.929	2	4	28	544.3
Android	GPT-4o	1	32.972	5	4	28	158.1
Android	GPT-4o	2	34.554	4	4	15	108.3
Android	DeepSeek-V3.2	1	34.498	11	4	18	415.7
Android	DeepSeek-V3.2	2	34.468	2	4	9	464.0
Android	Qwen3-32B	1	31.964	3	1	10	695.7
Android	Qwen3-32B	2	31.964	4	1	10	771.3
Android	Qwen3-Coder	1	29.452	16	4	28	542.3
Android	Qwen3-Coder	2	FAIL	15	—	28	535.9
Android	Gemini-2.5-Pro	1	31.967	1	0	4	551.4
Android	Gemini-2.5-Pro	2	31.967	1	0	4	552.3

Entries counts saved plan entries, including residual descriptors. It is not the number of effective generated programs. Active blocks counts held-out blocks emitting at least one semantic stream, out of four. Activity alone does not establish a compression benefit. All 35 valid ratios pass block-level and independent archive-only suffix decoding. The four zero-stream trials preserve bytes through residual compression. The failed Qwen3-Coder trial remains in the denominator, costs, and ledger.

Calls counts attempted requests, including missing responses. Synthesis duration includes model-service time and verification, excluding prefix extraction, sampling, storage fitting and online encoding. No failed trial is replaced by a new generation. The six Qwen3-Coder trials come from an earlier completed cohort, and the other thirty from a later one. Input, sample, first-prompt, and runtime/backend identities are audited before combining them.

13 Additional baselines and controls

All experiments in this section were specified in a design document written before any of their results existed. Later changes are appended to that document as dated change-log entries and are stated below. Every compression cell decodes from its archive alone and matches per-block and full-file SHA-256 digests of the original inputs.

13.1 Log-specific baselines

The +R correction deviates from the pre-registered adaptation rule (recorded in the change log). It was recorded before any new-baseline ratio comparison and was not applied to Denum. LogShrink (commit 59ce494) and LogReducer (commit 4000541) run their official pipelines independently on every 100,000-line block, including their own sampling and template training on that block (samplers seeded with 0). Their decoders read only the block archive, which contains the native payload, every model file the official restore reads, and, for the +R variants, a per-line correction. As released, both tools drop carriage returns, strip leading and trailing whitespace, collapse runs of header spaces, and drop leading zeros. LogShrink also truncates integers wider than 32 bits (e.g., HDFS block identifiers). The correction is computed from the original block and the tool’s own decoded output only, compressed with LZMA, and counted. LogShrink’s official decoder additionally needed five fixes that change no archive byte (empty failed-log files, column-file ordering, header-file ordering, whole-line template matching, and delimiter recovery). LogShrink uses its shipped per-dataset header lengths, while LogReducer uses its generic defaults. LogReducer+R failed the SHA check on 48 of Spark’s 333 blocks. LogShrink could not encode or decode 48 of Spark’s 333 blocks and 42 of Windows’s 1147 blocks. These files have no valid ratio for the affected tool. The official DeLog build (commit 64a074f) and DeLog-gen differ only in the per-dataset regular-expression map (emptied in DeLog-gen). The decoders are identical.

Table 24. Supplementary external measurements. The left part shows further general-purpose settings (Zstd19+D uses a dictionary trained on block 0 and counted once per file) and DeLog without its dataset-specific regular expressions, all byte-exact from archives alone. Zstd22L is `zstd -ultra -22 -long=27`. The right part shows compression-only ratios of LogShrink and LogReducer as released (not byte-exact, with exact files marked e), the number of LogShrink blocks that failed to encode or decode, and Denum’s compression-only ratio (its format cannot restore the input, so it is not losslessly verifiable).

Dataset	XZ6	Zstd3	Zstd22L	Zstd19+D	DeLog-gen	LS native	LS failed	LR native	LR status	Denum
Android	19.37	12.56	21.07	19.07	25.28	21.48	0	23.32	PASS	32.67
Apache	28.38	18.85	29.01	17.67	46.06	44.07 ^e	0	40.15 ^e	PASS	58.67
BGL	22.05	10.57	21.25	20.19	34.56	46.15	0	33.16	PASS	41.39
Hadoop	37.66	26.17	42.12	38.20	61.79	64.07	0	59.78	PASS	82.74
HDFS	15.63	10.27	15.83	14.50	28.33	24.90	0	23.37	PASS	25.32
HealthApp	15.83	9.92	16.35	15.14	46.56	32.34	0	14.80	PASS	45.27
HPC	19.81	9.79	16.98	16.11	44.60	40.04	0	32.43	PASS	44.90
Linux	19.97	12.24	19.56	10.58	24.03	28.39	0	27.38	PASS	30.87
Mac	23.86	16.95	26.06	22.25	38.69	38.24	0	39.28	PASS	40.17
OpenSSH	21.65	14.32	24.39	23.48	66.05	102.87	0	83.48	PASS	101.42
OpenStack	15.65	12.14	16.31	15.74	20.22	22.55	0	22.99 ^e	PASS	22.12
Proxifier	21.08	13.59	21.13	11.12	25.62	26.95	0	27.34	PASS	27.32
Spark	23.07	15.24	25.15	24.81	49.40	—	48	—	FAIL	59.66
Thunderbird	28.54	18.82	29.46	27.71	60.34	58.37	0	49.80	PASS	62.75
Windows	217.81	145.95	291.39	247.41	493.23	—	42	327.42	PASS	480.96
Zookeeper	29.63	23.03	38.80	27.64	116.67	111.87	0	116.39	PASS	135.40

Denum. The released Denum compressor (commit a3a6975) replaces each match of its per-dataset regular expressions by a tag and stores the concatenated digits as one integer, without group widths or separator positions. Its Python decoder also drops carriage returns and leading zeros. For each of the sixteen datasets we changed one line of block

0 by moving a digit across a group boundary. In all 16 cases the original and the modified block produce byte-identical archive members, so no decoder can restore both. Examples are [Thu Jun 09 06:07:04 2005] [notice] LD vs. [Thu Jun 09 0:607:04 2005] [notice] LD, ...3 R02-M1-N0-C:J12-U11 2005-06-03-15.42.50.363779 R vs. ...3 R02-M1-N0-C:J12-U11 200-506-03-15.42.50.363779 R, and ...08999687919862906 src:/10.250.19.102:54106 dest: vs. ...08999687919862906 src:/1.0250.19.102:54106 dest: . Denum is therefore excluded from the lossless comparison.

LogNexus (LogPrism as a preprint). The LogPrism preprint (arXiv 2601.17482) was published at ISSTA 2026 as LogNexus by the same authors, with an artifact on Zenodo (record 21021398, and the later record 21281836 changes only documentation, which we verified file by file). We built its released source unchanged (GCC 10 instead of the artifact’s GCC 11 container) and ran its default serial configuration (LogNexus_compress <block> <dataset> 100000 1 1 1 <tau>) independently on every 100,000-line block, with the per-dataset thresholds of its paper tables (0.02, its untuned default, for the four unseen sources, whose names select no built-in rules). Every file it writes is stored and counted, and its own decompressor reads only the block archive. Its artifact claims, and we confirm, restoration of the whitespace-separated token sequence only. It collapses whitespace runs and may omit blank lines. LogNexus+R adds a counted, dataset-agnostic, LZMA-compressed correction that holds the omitted blank lines with their positions, and for every other differing line the separators that differ when the token lists agree (the LogReducer+R line patch otherwise). This correction is cheaper for LogNexus than the line patch of the other +R variants. It was fixed after a probe on Linux only, before any other LogNexus result (change log). Reported ratios in the LogPrism preprint are listed for reference (mean 85.50× on the sixteen LogHub files). The accepted paper’s mean of 88.20× quoted in the main text is from its artifact (README and reference/paper_aggregate_results.csv).

Table 25. LogNexus as released (native, not byte-exact) and with the counted correction (+R, byte-exact from the archive alone), next to SemZip. The Token column counts blocks whose restored token sequence equals the original. Corr. is correction bytes as a share of the +R archive. Preprint is the ratio reported in LogPrism v2 (inputs unverified).

Dataset	τ	Native	Token	+R	Corr.	Preprint	SemZip
Android	0.05	32.41	16/16	28.61	11.7%	32.82	34.55
Apache	0.02	65.31	1/1	65.24	0.1%	64.33	73.74
BGL	0.005	47.65	48/48	47.65	0.0%	47.15	52.32
Hadoop	0.005	80.61	4/4	77.17	4.3%	79.73	85.03
HDFS	0.005	25.91	112/112	25.32	2.3%	26.47	31.05
HealthApp	0.005	49.45	3/3	47.05	4.9%	50.48	59.53
HPC	0.01	45.99	5/5	45.50	1.1%	45.45	45.79
Linux	0.001	38.21	1/1	36.67	4.0%	38.34	40.09
Mac	0.12	43.70	2/2	39.87	8.8%	44.99	54.13
OpenSSH	0.005	110.89	7/7	101.72	8.3%	113.33	117.78
OpenStack	0.001	23.28	3/3	23.28	0.0%	23.36	25.24
Proxifier	0.01	30.07	1/1	27.61	8.2%	29.85	36.13
Spark	0.001	54.70	333/333	51.68	5.5%	54.81	62.35
Thunderbird	0.002	79.27	2113/2113	79.27	0.0%	75.86	64.86
Windows	0.1	538.93	1147/1147	509.51	5.5%	497.30	618.58
Zookeeper	0.001	144.42	1/1	132.59	8.2%	143.74	189.42
NASA	0.02	21.13	19/19	21.12	0.0%	–	22.76
ClarkNet	0.02	15.89	17/17	15.89	0.0%	–	19.12
USask	0.02	15.95	25/25	15.95	0.0%	–	16.65
Calgary	0.02	19.89	8/8	19.86	0.1%	–	21.12

Published numbers without an implementation. LogFold (ICSE 2026) provides no public implementation (checked on 2 October 2026, when its repository held only a license and README). Its paper reports the following ratios for 100,000-line chunks. Input identities are not verified and these numbers are not matched observations.

Table 26. Published ratios (literature context only) next to SemZip.

Dataset	LogFold (ICSE 2026, arXiv 2603.20618v1)	SemZip
Android	32.72	34.55
Apache	57.81	73.74
BGL	51.02	52.32
Hadoop	90.97	85.03
HDFS	25.22	31.05
HealthApp	56.56	59.53
HPC	46.95	45.79
Linux	31.51	40.09
Mac	49.40	54.13
OpenSSH	96.76	117.78
OpenStack	20.39	25.24
Proxifier	29.14	36.13
Spark	59.88	62.35
Thunderbird	67.69	64.86
Windows	570.55	618.58
Zookeeper	145.74	189.42

Released DeLog implementation versus published ratios. Running the released DeLog (commit 64a074f) on whole files with the command line of its own benchmark script (text mode, 100,000-line blocks, four threads, frequency threshold 0, LZMA, normal mode) gives $27.60\times$ on Linux and $40.33\times$ on BGL, identical to our block adapter, whereas the DeLog paper reports $30.63\times$ and $45.68\times$. All tables use the measured values. The published ratios are listed here.

Table 27. DeLog measured on our inputs versus published (literature only).

Dataset	Measured	Published	Difference	SemZip
Android	29.29	30.35	+3.6%	34.55
Apache	59.77	59.65	-0.2%	73.74
BGL	40.33	45.68	+13.3%	52.32
Hadoop	80.60	79.63	-1.2%	85.03
HDFS	29.83	29.01	-2.7%	31.05
HealthApp	51.40	55.93	+8.8%	59.53
HPC	44.70	46.84	+4.8%	45.79
Linux	27.60	30.63	+11.0%	40.09
Mac	44.22	43.69	-1.2%	54.13
OpenSSH	107.76	106.01	-1.6%	117.78
OpenStack	21.70	23.75	+9.4%	25.24
Proxifier	30.19	30.78	+2.0%	36.13
Spark	59.05	61.79	+4.6%	62.35
Thunderbird	67.68	68.65	+1.4%	64.86
Windows	536.18	541.57	+1.0%	618.58
Zookeeper	156.89	154.93	-1.3%	189.42

13.2 Program-library control

The library proposer replaces only the LLM call of the trainer. Compilation, verification, the quality gate, storage fitting, the runtime, and archive-only decoding are unchanged, and it makes no API call. Its program types were frozen before any library result. They cover syslog, ISO-8601, YYYY-MM-DD HH:MM:SS (with optional milliseconds), YY/MM/DD, YYYYMMDD HHMMSS, clock-time, and ten-digit epoch timestamps, plus IPv4 and IPv4:port, hexadecimal numbers, sizes with B/KB/MB/GB units, percentages, and decimal integers and fixed-point numbers with width and leading zeros. A type is proposed for a sampled group when it matches one of the group’s examples (at most eight per group). With

one candidate per dataset, pooled selection reduces to the gated plan. The gate removes one rule group at a time and only on a strict decrease, and never compares with the empty program. On HealthApp, where no library timestamp matches, fourteen generic integer rules leave block 0 3.4 times larger than the empty program. The post-hoc floor (not pre-registered) deploys the empty program when it is smaller on block 0. SemZip’s deployed plans are below the empty program on block 0 for all sixteen files.

Table 28. Program-library control with block-0 payload bytes and complete-file ratios.

Dataset	Library B0	Empty B0	Library	Lib.+floor	SemZip
Android	493,280	410,416	27.51	31.83	34.55
Apache	101,084	120,736	50.73	50.73	73.74
BGL	400,556	418,044	33.79	33.79	52.32
Hadoop	384,372	495,324	64.90	64.90	85.03
HDFS	582,564	508,312	22.10	24.03	31.05
HealthApp	739,028	216,824	12.50	39.85	59.53
HPC	479,204	194,936	15.75	44.79	45.79
Linux	79,392	92,204	29.54	29.54	40.09
Mac	381,416	316,776	35.03	42.07	54.13
OpenSSH	264,948	152,356	37.16	66.19	117.78
OpenStack	1,768,572	1,393,624	17.73	22.45	25.24
Proxifier	115,932	96,280	21.90	26.36	36.13
Spark	389,208	288,084	39.71	50.91	62.35
Thunderbird	482,248	293,036	37.39	64.65	64.86
Windows	49,236	57,652	502.31	502.31	618.58
Zookeeper	135,160	92,096	77.05	113.02	189.42

13.3 Matched replay on the deployed plans

Both branches execute the deployed matcher once per block and share its spans, rule order, placeholders, residual archive (byte-identical), and semantic XZ preset. The surface branch stores each replaced literal with identity reconstruction and generic field storage and fits its own block-0 storage policy. The latent branch reproduces the formal SemZip archives byte for byte. Large files use the twenty blocks round($i(N - 1)/19$).

Table 29. Matched replay with surface and latent archive bytes.

Dataset	Scope	Blocks	Surface	Latent	Saving
Android	complete	16/16	5,602,902	5,565,790	0.66%
Apache	complete	1/1	82,026	69,646	15.09%
BGL	complete	48/48	14,680,374	14,203,694	3.25%
Hadoop	complete	4/4	633,820	571,500	9.83%
HealthApp	complete	3/3	433,917	395,273	8.91%
HPC	complete	5/5	732,843	732,843	n/a
Linux	complete	1/1	63,725	58,613	8.02%
Mac	complete	2/2	313,549	311,853	0.54%
OpenSSH	complete	7/7	728,835	623,323	14.48%
OpenStack	complete	3/3	2,481,449	2,434,257	1.90%
Proxifier	complete	1/1	79,389	70,361	11.37%
Zookeeper	complete	1/1	63,013	55,045	12.65%
HDFS	sample20	20/112	10,051,953	10,051,953	n/a
Spark	sample20	20/333	2,887,699	2,876,067	0.40%
Windows	sample20	20/1147	767,784	773,208	-0.71%
Thunderbird	sample20	20/2113	3,911,949	3,911,949	n/a

13.4 Complete reruns

Each rerun draws five fresh syntheses (greedy and four at temperature 0.7) from the same GPT-4o gateway with the unchanged trainer and budgets, then gates, selects, fits storage, and encodes and decodes the complete file. Run 1 is the reported deployment and is not replaced.

Table 30. Complete reruns with ratios and archive bytes relative to DeLog.

Dataset	Run	Ratio	Suffix ratio	vs. DeLog	Suffix vs. DeLog
HPC	1	45.79	45.03	−2.37%	−2.73%
HPC	2	45.73	45.18	−2.25%	−3.06%
HPC	3	46.79	45.89	−4.45%	−4.56%
HDFS	1	31.05	31.00	−3.93%	−3.93%
HDFS	2	31.50	31.45	−5.30%	−5.29%
HDFS	3	31.05	31.00	−3.93%	−3.93%
Hadoop	1	85.03	109.25	−5.22%	−3.65%
Hadoop	2	84.53	108.79	−4.65%	−3.23%
Hadoop	3	87.82	114.14	−8.23%	−7.77%
Spark	1	62.35	62.41	−5.29%	−5.29%
Spark	2	61.71	61.75	−4.31%	−4.27%
Spark	3	62.53	62.57	−5.57%	−5.52%

13.5 Decode-time safety

Audit. The runtime executes `python_exec` code with `exec(compile(...))` after an allow-list check of syntax nodes, in a namespace limited to `datetime`, `timedelta`, `int`, `str`, `len`, `float`, `round`, `abs`, `min`, `max`, and a restricted import shim. Context programs also receive the `re` module. Names beginning with a double underscore are rejected. Code runs in the decoding worker process with the decoder’s privileges and no resource limits, and semantic archives are unpacked without member-path filtering. Four crafted archives derived from the Apache archive test this boundary. They are an injected import (rejected by the runtime), a format string reading `{0.__class__}` (accepted), a context program reading `re.enum.sys` (accepted, top level executed), and a tar member `../path` (not executed against the unprotected decoder).

Check. The enforced check rejects imports, names and attributes beginning with an underscore outside the runtime’s fixed wrapper, reflective built-ins, format fields, and module attributes outside an allow-list, and preflights tar members. It rejects all four crafted archives. Over 89 distinct programs from all syntheses, deployments, and evolution runs, the only strict-rule findings (144 underscore names in 12 programs) come from the runtime’s own fixed wrapper, which the check recognizes exactly. LLM-written code has no violation. A later pass with the same validator over the repeated greedy syntheses, the complete reruns, and the unseen-source runs adds 45 distinct LLM-written programs (134 in total) and again finds no violation outside the fixed wrapper. The eleven hand-written library programs also pass. With the check enforced before any program is compiled, all sixteen archives (3,797 blocks) decode byte-exactly.

13.6 Unseen sources

The four logs are the LBL Internet Traffic Archive HTTP access logs NASA (July 1995), ClarkNet (28 August 1995), USask, and Calgary, selected by a fixed rule (every server log with at least 700,000 lines, one file per server) before any result. They are decompressed unchanged. The SemZip pipeline differs from the main runs only in reading the input identities of these files. LogShrink+R (default header length) fails on ClarkNet (its decoder crashes on nine blocks containing bare carriage returns) and on USask (its encoder fails on block 2). LogReducer+R fails on USask (the official encoder crashes on block 2). On the 24 USask blocks both tools encode, LogShrink+R is 1.3% smaller and LogReducer+R 0.2% larger than SemZip on the same blocks.

Table 31. Unseen sources. Complete-file ratios of every method except LogNexus+R (Table 25), and suffix ratios (blocks 1 onward) of SemZip and DeLog. A dash means the tool failed on at least one block.

Source	SemZip	DeLog	LS+R	LR+R	LogLite	gzip6	XZ6	XZ9e	Zstd3	Zstd19	Suffix S/D
NASA	22.76	20.31	21.56	18.96	9.82	9.92	15.10	16.36	9.72	15.59	22.79/20.32
ClarkNet	19.12	17.04	–	15.38	9.14	7.85	13.82	14.44	9.31	13.70	19.13/17.05
USask	16.65	15.28	–	–	8.63	7.65	13.13	13.60	8.59	12.86	16.67/15.29
Calgary	21.12	18.66	21.23	21.25	12.36	9.61	15.51	16.87	9.44	15.13	20.98/18.54

13.7 Large-file throughput

The four larger files were timed on twenty systematically spaced original blocks each (block indices $\text{round}(i(N-1)/19)$, byte-identical to the originals), three clean serialized trials per method and file with four block workers, in a session separate from the twelve-file session. Median MB/s of original input. A dash means not losslessly restored in all repeats (timed only).

Table 32. Large-file throughput per sampled file (encode/decode MB/s).

Method	HDFS		Spark		Windows		Thunderbird	
SemZip	7.67	49.83	5.46	24.23	9.58	57.20	11.86	55.77
DeLog	26.77	75.66	24.29	56.54	43.83	124.64	20.08	66.53
LogLite-BL*	19.80	85.96	27.77	76.74	71.60	153.54	30.16	102.58
gzip6	87.26	182.59	65.89	126.68	131.07	220.26	85.84	156.45
XZ6	9.95	129.57	12.02	101.85	40.94	218.63	14.76	136.91
Zstd3	142.09	196.50	91.14	137.80	175.82	279.50	121.88	172.50
XZ9e	3.77	108.12	2.36	91.60	7.81	200.32	1.71	132.65
Zstd19	5.27	157.36	4.61	100.56	22.51	241.82	4.07	129.73
LogShrink+R*	1.22	2.93	–	–	–	–	0.54	1.26
LogReducer+R*	3.36	11.38	–	–	4.40	14.47	0.52	0.63

14 Example proposer output

An illustrative candidate program for a rendered byte value (expanded for readability).

Listing 1. Representative proposer output for a rendered byte value. The embedded program code is expanded for readability.

```

candidate = {
  "tag": "SIZE",
  "meaning": "byte count with a KB view",
  "class": "class_1_composed_numeric",
  "regex": r"(\d+)( +)bytes( +)\((\d+\. \d+)( +)KB\)",
  "replacement": "{placeholder}",
  "program": {"op": "python_exec",
    "code": "<expanded below>"}
}

# candidate["program"]["code"]
def forward(groups):
    raw, sp1, sp2, shown, sp3 = groups
    precision = len(shown.rsplit(".", 1)[1])
    return {"stored": [int(raw)],
      "layout": [len(sp1), len(sp2),

```

```

len(sp3), precision]]}

def inverse(record):
    b = int(record["stored"][0])
    sp1, sp2, sp3, p = record["layout"]
    scale = 10 ** p
    shown = (b * scale + 512) // 1024
    shown = f"{shown // scale}.{shown % scale:0{p}d}"
    return (f"{b}{' ' * sp1}bytes{' ' * sp2}"
            f"({shown}{' ' * sp3}KB)")

```

15 Program synthesis, repair, and verification loop

Algorithm S1 Counterexample-guided program synthesis, repair, and verification

Input log-group examples E , masks and contexts C , program schema Σ , repair budget B

Output verified program set V

```

 $Q \leftarrow \text{ProposerAgent}(E, C, \Sigma)$ 
 $V \leftarrow \emptyset, b \leftarrow 0$ 
while  $Q \neq \emptyset$  do
     $p \leftarrow \text{Pop}(Q)$ 
     $(\hat{p}, d_c) \leftarrow \text{Compile}(p, \Sigma)$ 
    if  $\hat{p} = \perp$  then
         $d \leftarrow d_c$ 
    else
         $(ok, d) \leftarrow \text{Verify}(\hat{p}, E, \Sigma)$ 
        if  $ok$  then
             $V \leftarrow V \cup \{\hat{p}\}$ 
            continue
    if  $b < B$  and  $\text{Repairable}(d)$  then
         $R \leftarrow \text{RepairAgent}(p, d, E, C, \Sigma)$ 
         $Q \leftarrow Q \cup R$ 
         $b \leftarrow b + 1$ 
return  $V$ 

```

16 Complete-file records of online evolution

Chronology. The protocol was designed after the main comparison (Thunderbird was the only file larger than DeLog’s) and after the historical single-synthesis cohorts below. Its numeric constants were recorded in a design note before any run. Three implementation clarifications (trigger window from block 1, publication-gap anchoring, and requiring a served block) were added after the program-free measurements and before any evolution run, and nothing was changed afterwards. It was first screened on Thunderbird blocks 0–99. The prespecified rule extended it to the full file when blocks after the first publication shrank by at least 1% relative to the frozen deployment with the program-free choice (they shrank by 22.2%). Running it unchanged on Spark was decided after that window result and before any Spark update. The other files were decided from the trigger replay below.

Protocol. A block is a *loss* when the best published program is not at least 1% smaller than the empty program (residual processing only) on that block. An update triggers when at least 3 of the last 5 completed blocks (from block 1) are losses, at least 10 blocks after the latest publication (the initial deployment counts as block 0, and an update counts from the block it serves first), not within a 10-block cooldown after a rejected or failed attempt, and only if block $b + 2$ exists, so that a publication serves at least one block (files with fewer than 13 blocks cannot trigger). Each update runs the unchanged training pipeline (one greedy and four temperature-0.7 fresh syntheses, gate, cost-guided selection, storage fit) on the trigger block b . Already published programs are not re-pooled. The candidate is encoded on block $b + 1$ through the same semantic-archive guard and must be at least 1% smaller than the minimum over published programs and the empty program. It serves blocks from $b + 2$. At most 5 publications and 8 attempts. Every block

stores the smallest payload among the published programs and the empty program. Blocks are self-describing, and assembled archives are decoded archive-only. Time is counted in blocks. Synthesis latency is not modeled.

Table 33. Trigger replay and program-free choice on every file. Loss blocks are blocks from 1 on where the frozen program is not at least 1% smaller than the empty program. First trigger is the first block at which the rule fires for the frozen deployment, n/a when fewer than 13 blocks mean the rule cannot fire, and – when the file is eligible but the rule never fires. Ratios are given for the frozen deployment and the per-block minimum of frozen and empty program, assembled and decoded archive-only (SHA-256 match) on every file where they differ. Empty s gives the wall seconds of the full-file empty-program encode (four workers, shared host), the monitoring cost.

Dataset	Blocks	Loss blocks	First trigger	Frozen	Frozen+empty	Empty s
Linux	1	0	n/a	40.09	40.09	1
Proxifier	1	0	n/a	36.13	36.13	2
Apache	1	0	n/a	73.74	73.74	2
Zookeeper	1	0	n/a	189.42	189.42	2
Mac	2	0	n/a	54.13	54.13	3
HealthApp	3	0	n/a	59.53	59.53	3
HPC	5	2	n/a	45.79	46.08	3
Hadoop	4	0	n/a	85.03	85.03	3
OpenStack	3	0	n/a	25.24	25.24	6
OpenSSH	7	0	n/a	117.78	117.78	5
Android	16	0	–	34.55	34.55	11
BGL	48	0	–	52.32	52.32	56
HDFS	112	0	–	31.05	31.05	82
Spark	333	68	21	62.35	62.48	175
Windows	1147	5	–	618.58	618.70	802
Thunderbird	2113	1662	13	64.86	65.06	1,358

Table 34. Every update attempt. Bytes are given for the candidate and the current minimum (published programs and empty program) on validation block $b + 1$. Seconds give synthesis, gate, selection and publication wall time. Spark blocks 32–86 consist of at least 90% non-Spark **Update row** records (98.9–100.0% on the six rejected attempt blocks). The six rejected candidates share a byte-identical empty extraction program with separately fitted storage policies. Zero LLM calls means the block yielded no example group eligible for the proposer. Candidate encodes are cached per bank slot, so the last five reuse the first rejected candidate’s validation encode. All six equal the current minimum.

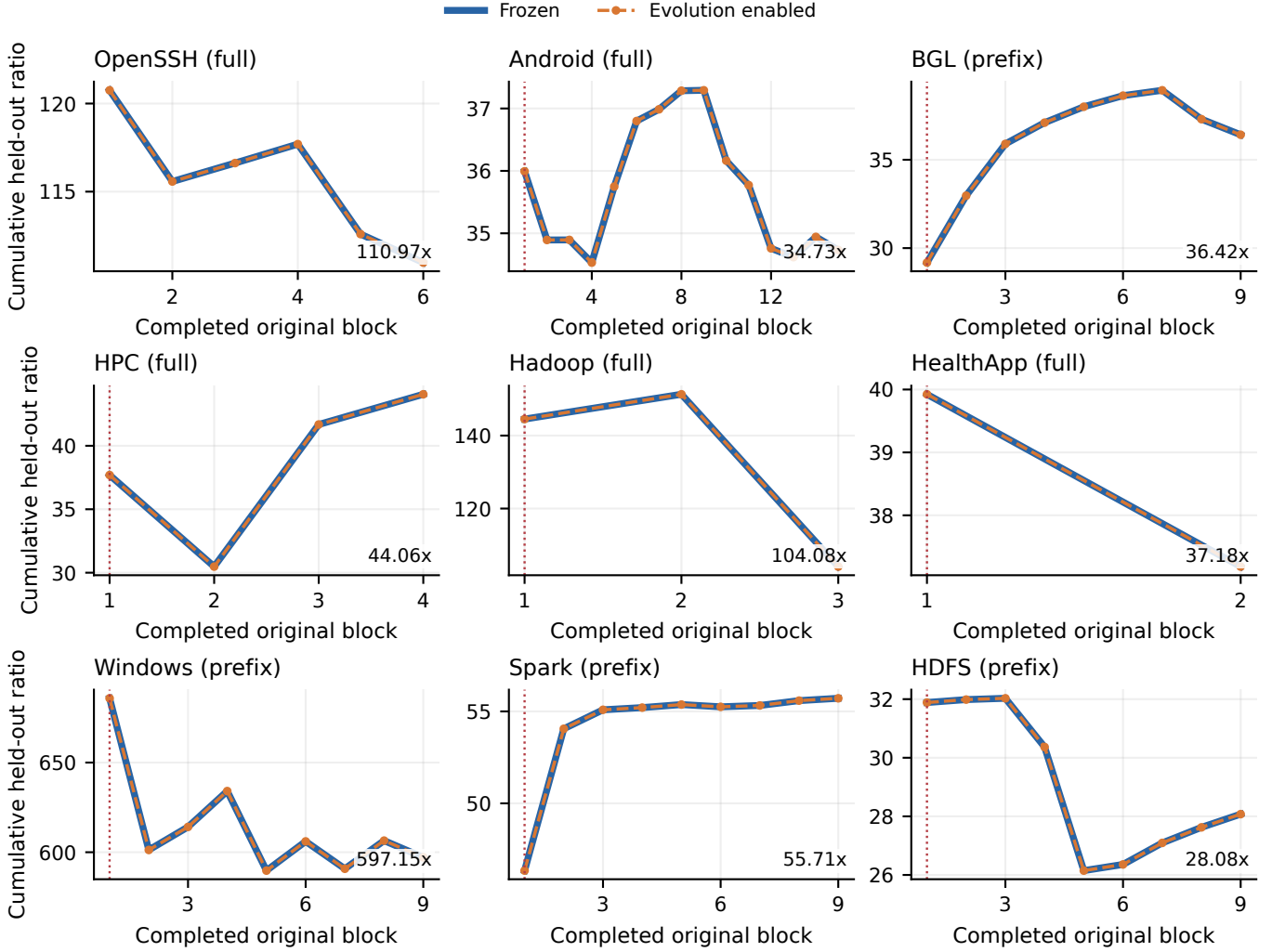
Dataset	Block b	LLM calls	Seconds	Candidate	Current min	Outcome
Thunderbird	13	27	2213	140,032	176,212	published from 15
Spark	21	36	1554	146,836	155,932	published from 23
	34	0	11	277,512	277,512	rejected
	44	0	11	278,568	278,568	rejected
	54	11	120	277,696	277,696	rejected
	64	10	109	277,528	277,528	rejected
	74	0	11	277,632	277,632	rejected
	84	0	11	277,652	277,652	rejected

Table 35. Complete-file outcomes of the triggered files. The first five rows are compression ratios. Evolution is the per-block minimum over published programs and the empty program (assembled archive, archive-only decode, SHA-256 match). Newest only retains just the newest program after each publication. Later covers blocks from the first served block on. Encode seconds give the additional wall time of the evolution runs, summed over the 100-block part encodes, on the shared host concurrently with other jobs, not in the serialized timing session. Frozen-program payloads are reused from the formal run. The new Thunderbird program is 16.1% smaller than the frozen one on block 0 (237,788 vs 283,484 bytes).

Quantity	Thunderbird	Spark
Frozen deployment	64.86	62.35
Frozen + empty program	65.06	62.48
Evolution	78.66	63.29
Evolution, newest only	78.66	63.12
DeLog	67.68	59.05
First served block	15	23
Later bytes vs frozen+empty	-17.41%	-1.40%
Later bytes vs DeLog	-14.07%	-6.77%
Blocks smaller than DeLog	2,070/2,113	282/333
Assembled archive bytes	404,119,462	46,472,727
Program-free encode seconds	1,572	210
Published-program encode seconds	5,112	927
Rejected-candidate encode seconds	0	34
Frozen first-pass encode seconds	8,537	2,076

A fallback to upstream DeLog chunks was not evaluated as a SemZip variant because it would import DeLog’s per-dataset recognizer map. The historical single-synthesis evolution cohorts follow unchanged in the next sections.

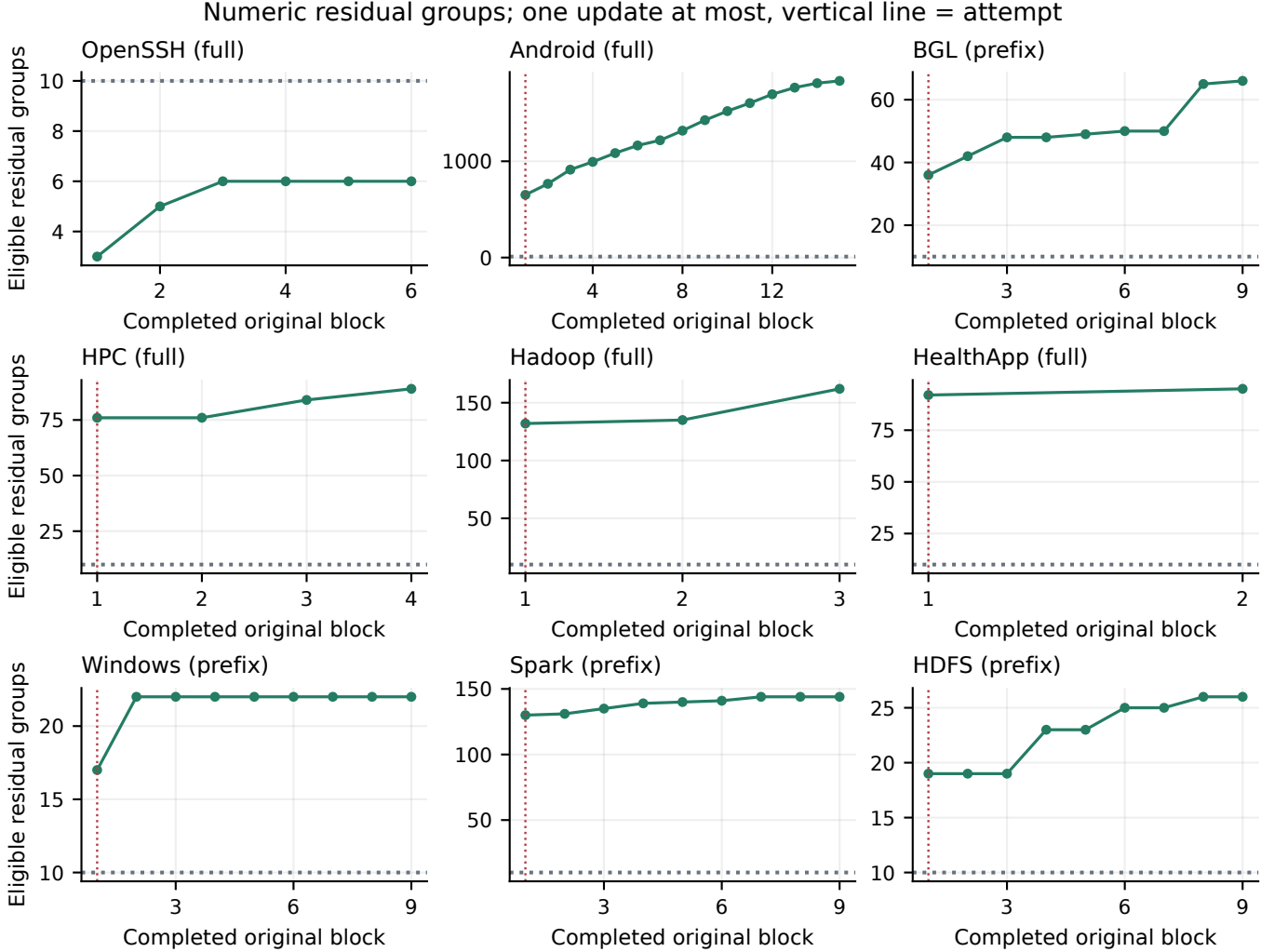
17 Historical single-synthesis evolution pilots



Dataset	Scope	Blocks	Held-out ratio	Updates	Archive saving
OpenSSH	complete file	7	110.9720	0	0.0000%
Android	complete file	16	34.7325	1	0.0000%
BGL	first 10 blocks	10	36.4155	1	0.0000%
HPC	complete file	5	44.0569	1	0.0000%
Hadoop	complete file	4	104.0819	1	0.0000%
HealthApp	complete file	3	37.1842	1	0.0000%
Windows	first 10 blocks	10	597.1474	1	0.0000%
Spark	first 10 blocks	10	55.7134	1	0.0000%
HDFS	first 10 blocks	10	28.0751	1	0.0000%

Block counts include training block 0, while ratios and curves exclude it. All 75 paired blocks reconstruct, and both branches independently decode the evaluated file/prefix from archives. Paired archives are identical. Curve fluctuations reflect the changing input composition and do not show update gains. The six additional datasets were selected before observing their outcomes, by length and system diversity. Linux/Zookeeper are shorter than one full block, and Mac has no evaluation block after the earliest permitted update.

18 Historical single-synthesis residual novelty, triggering, and update cost

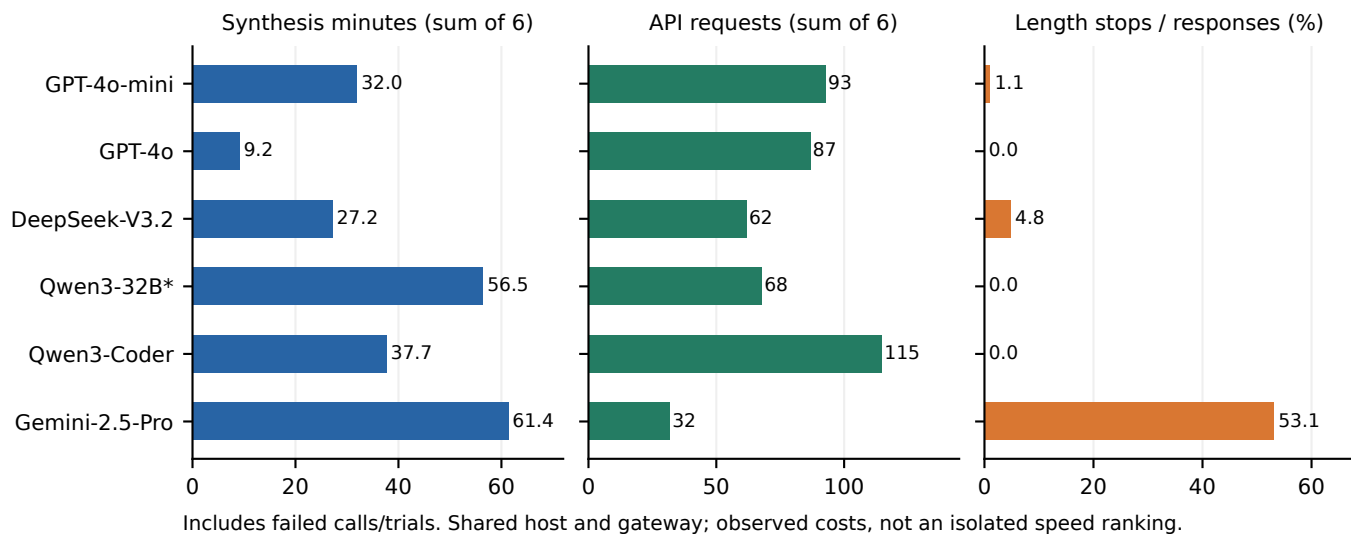


Dataset	Update requests	Update seconds	Published rules	Exact archive savings
OpenSSH	0	0.00	0	0 bytes
Android	3	51.66	0	0 bytes
BGL	7	52.75	0	0 bytes
HPC	3	31.84	0	0 bytes
Hadoop	4	57.78	0	0 bytes
HealthApp	5	36.58	0	0 bytes
Windows	4	47.80	0	0 bytes
Spark	17	86.54	0	0 bytes
HDFS	3	30.28	0	0 bytes

A group requires fifty supporting lines and two distinct values. Ten eligible numeric-residual groups trigger the single permitted update, using two original past records per group and at most forty groups. OpenSSH never triggers. Every other pilot attempts an update after block 1. The eight attempts use 46 calls and 395.24 seconds. Accumulating novelty after an unsuccessful attempt does not trigger a second update under this protocol.

The legacy publisher appends only new tags and preserves previous rules. Same-tag refinements are excluded. An empty publication does not mean every candidate failed compilation. This is a null result for the evaluated narrow detector, publisher, and time windows, not a universal statement about evolution.

19 Observed synthesis cost and historical default controls



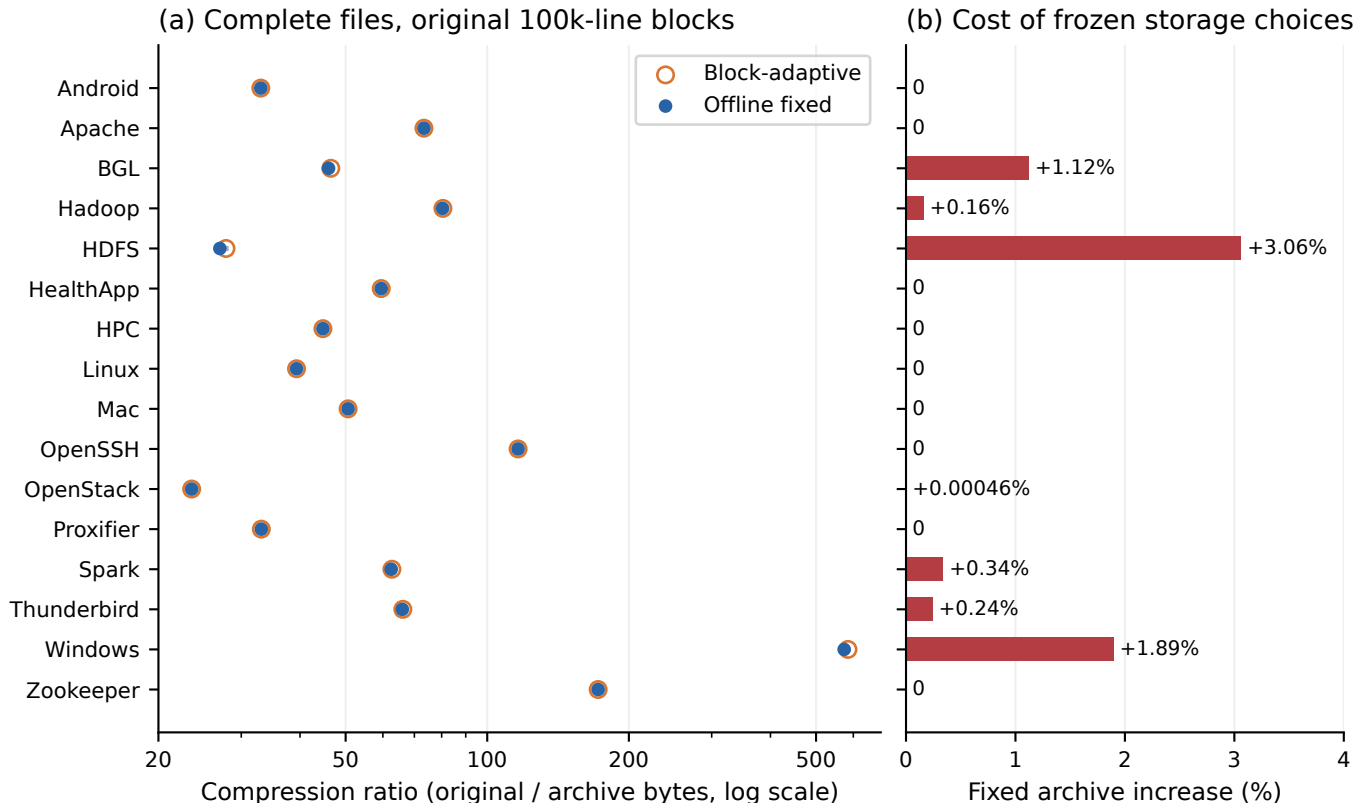
The explicit-allowance cohort records 457 requests and 452 responses. Four Qwen3-32B requests and one Qwen3-Coder request have no response. Gemini has 17 length stops in 32 responses, DeepSeek 3/62, and GPT-4o-mini 1/93. The other three conditions have none. Token accounting varies by provider. Reasoning tokens may be included in completion tokens or reported separately. Costs are observed on a shared host and gateway, not an isolated latency or monetary comparison.

All eighteen historical gateway-default trials. The request omits `max_tokens`, so do not pool these with the explicit-allowance cohort for model ranking.

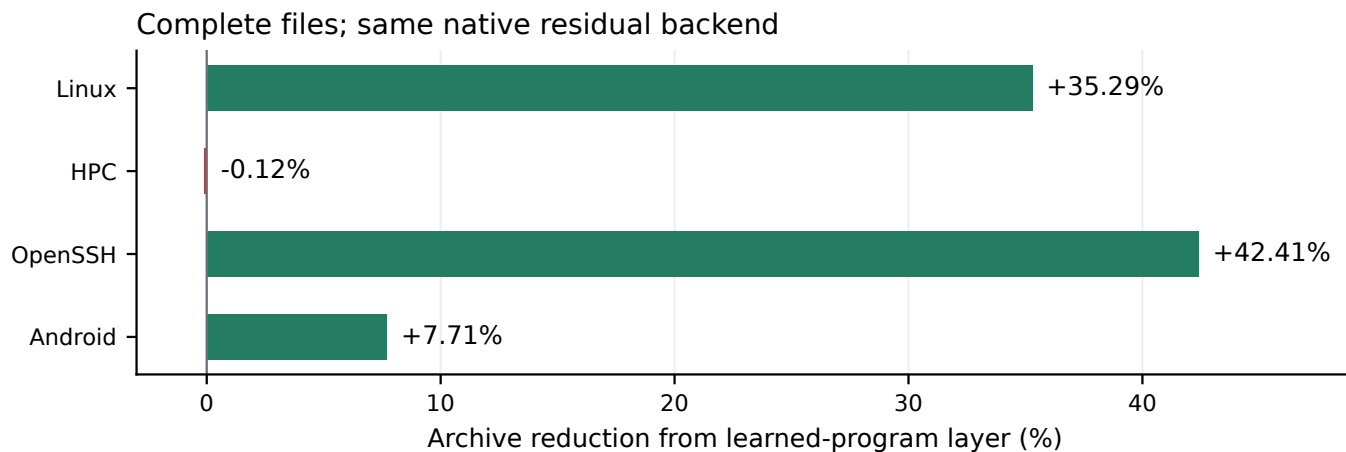
Dataset	Model	Run	Ratio	Calls	Synthesis s
HPC	GPT-4o	1	43.963	10	46.8
HPC	GPT-4o	2	44.052	14	61.0
HPC	DeepSeek-V3.2	1	45.281	8	153.9
HPC	DeepSeek-V3.2	2	45.111	8	204.2
HPC	Qwen3-Coder	1	44.086	5	57.4
HPC	Qwen3-Coder	2	44.466	5	60.9
OpenSSH	GPT-4o	1	FAIL	11	81.4
OpenSSH	GPT-4o	2	113.638	7	59.9
OpenSSH	DeepSeek-V3.2	1	113.466	21	288.1
OpenSSH	DeepSeek-V3.2	2	95.776	15	225.9
OpenSSH	Qwen3-Coder	1	65.516	5	50.2
OpenSSH	Qwen3-Coder	2	65.516	5	63.2
Android	GPT-4o	1	34.557	13	87.6
Android	GPT-4o	2	32.972	4	69.0
Android	DeepSeek-V3.2	1	34.442	25	420.8
Android	DeepSeek-V3.2	2	34.725	28	583.4
Android	Qwen3-Coder	1	31.967	4	58.7
Android	Qwen3-Coder	2	31.967	4	54.1

This separate cohort contains seventeen complete-protocol successes and one layout/codec failure. Default Qwen stops 27/28 responses by length at an observed maximum of 1,000 completion tokens. Four empty-stream helper errors use the unchanged saved plans and ordinary empty fixed policy, with no new API calls. Across both cohorts, there are 54 unique trainings, 52 complete-protocol successes and two retained failures.

20 Full-file storage and program-layer controls

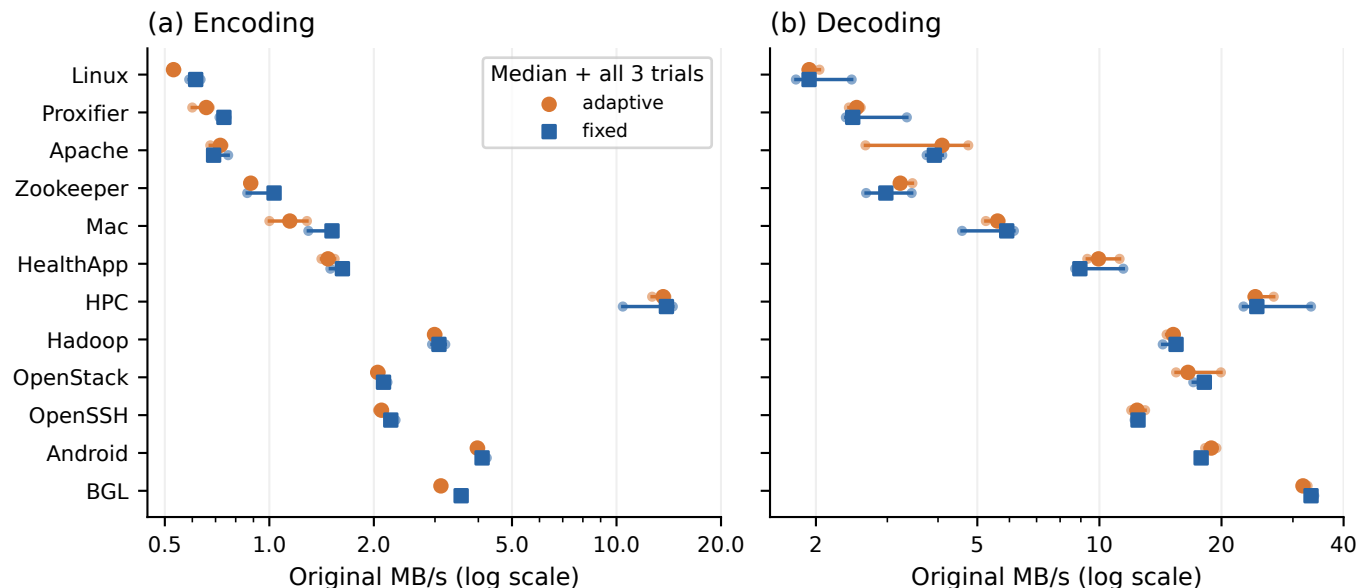


Sixteen complete-file pairs hold input bytes and saved extraction programs fixed. Nine archives are identical, while fixed storage increases total archive bytes by 0.6260%. Arithmetic-mean dataset ratios are $93.8215 \times$ fixed and $94.6147 \times$ adaptive. These historical plans are not fresh held-out backbone training, and the prefix results above do not replace Section 9.

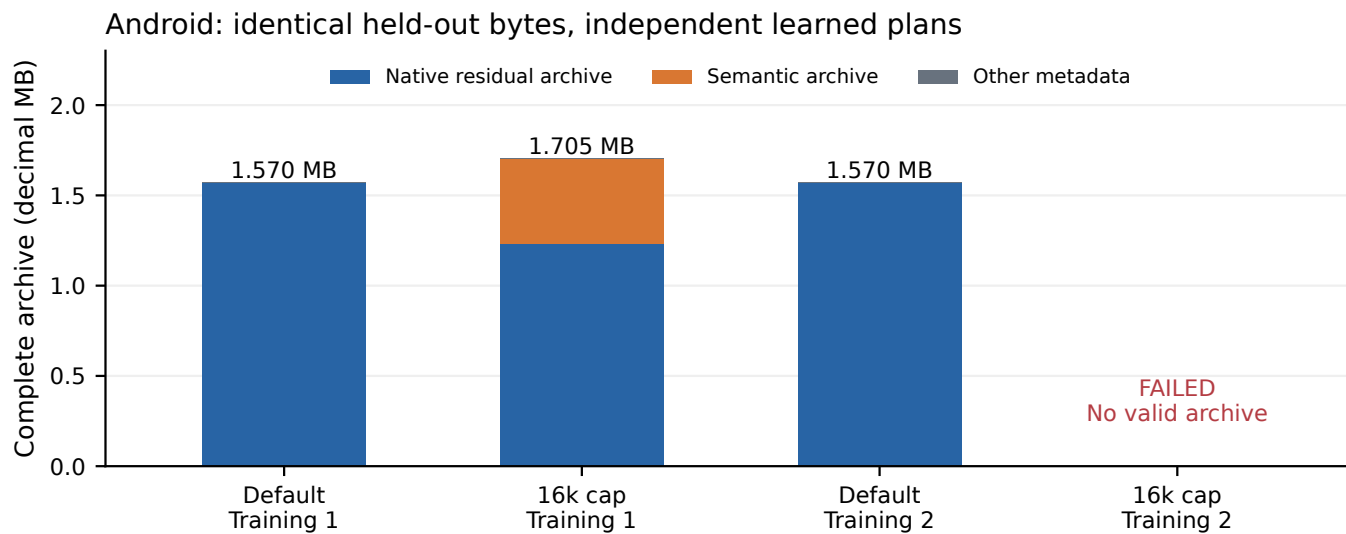


The learned-layer control uses the same native residual backend for both branches. Linux/HPC use historical plans, while OpenSSH/Android use fresh first-block plans. All eight complete files reconstruct. This measures the learned representation plus its storage, not LLM superiority over equivalent manually written programs. HPC's negative result is retained.

21 Throughput and archive-cost diagnostics



Twelve files no larger than BGL receive three alternating formal pairs after a first-pass warmup. Small points show all trials, large points show medians, and bars show ranges rather than confidence intervals. Four workers share an eight-CPU developer host. Encoding medians improve on eleven files and regress on Apache, and five ranges overlap. No cross-host speedup or significance is claimed.



The first Android default-Qwen trial has 1,472 semantic, 1,568,748 residual and 245 metadata bytes. Explicit allowance changes these to 472,320, 1,232,016 and 245 bytes. The archive grows 8.54% despite shorter transformed text. This is a plan-level diagnostic across independent generations, not a per-rule causal ablation. Failed reconstruction has no invented archive-size bar.

22 Failure boundaries and reproduction scope

Layout/codec mismatch. The historical default GPT-4o/OpenSSH plan contains an inverse requiring layout metadata, but its selected decode route omits that metadata. Sample validation passes, but full training-block reconstruction fails. The saved failure is not repaired or replaced in the reported cohort.

Literal-placeholder collision. The explicit-allowance Qwen3-Coder/Android run 2 passes first-block fitting, then fails on the next original block. Nine literal <T> strings collide with the saved placeholder, whose value stream is empty. A zero-API replay reproduces the failure. Successful evaluated decodes are not a proof over all possible input bytes.

Experiment helper corrections. Missing directories for an empty semantic stream are handled by the ordinary empty fixed policy, using the same saved plan. The four older model-control corrections and the HPC evolution correction retain original failures and request records. The HPC correction uses no additional initial-synthesis calls, but its normal update stage still makes three calls. Runtime/codec defects above remain unfixed in the tested source.

Deployment. Programs and storage recipes are learned offline and fixed within each deployment stage. The optional evolution protocol uses only completed-block evidence and may publish additional programs for future blocks, and earlier programs remain available. Program reuse across stages is distinct from cross-block compression state. Dictionaries, file aliases and differences remain block-local. The experiment trains programs, and LLM weights are not updated.

Evidence boundaries. Main results combine separate cohorts with explicit provenance. Backbone prefixes, complete-file storage pairs, program controls, and chronological pilots answer different questions. Public data may not reflect private production logs. Two trainings per model/dataset do not justify significance tests or a universal model hierarchy. The new matched external-system experiment belongs to a separate main cohort. Its complete-file sizes and three-repeat formal timings are reported separately, and published competitor numbers are never inserted as same-condition observations. The historical tables here retain the original runtime and failures, even when a new guard subsequently recovers the same input.

Reproduction. The evidence archive retains input/result hashes, frozen plans, sanitized model exchanges and source identities. Its `historical_evidence/` subtree preserves complete historical outcomes, while `historical_figures/render.py` generates all historical manuscript plots from package-local numeric inputs, with vector PDFs, PNGs and point-level tables. Rendering does not rerun synthesis, codecs or original certificate checks. The manuscript source package contains compilation inputs, while an anonymous evidence archive is prepared separately. No public access or later publication commitment is asserted here.

23 Why these backbone axes

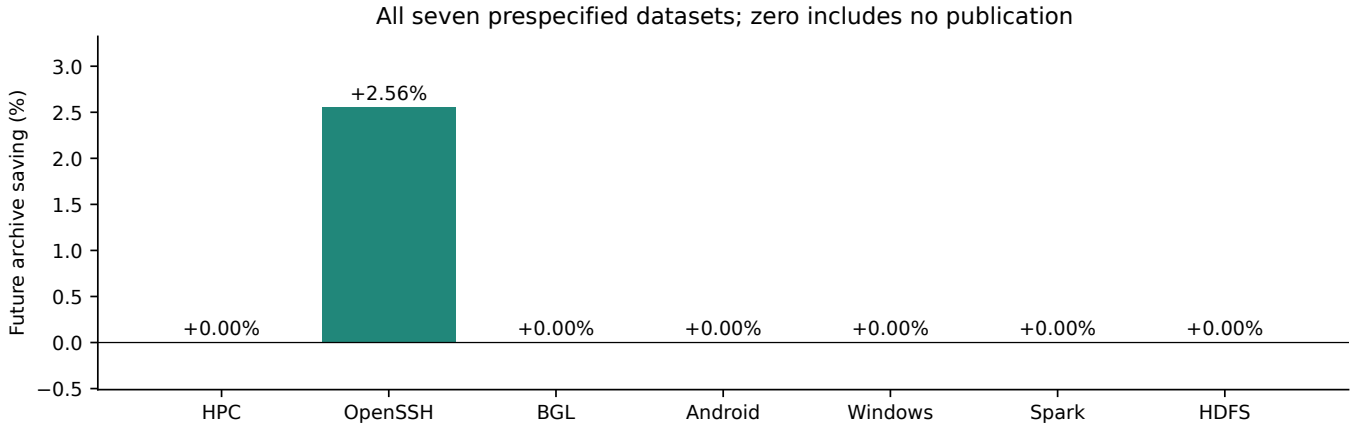
Relevant main-conference examples vary model family, scale, openness and search budget rather than imposing a universal model count. Programming by Example (NeurIPS 2024) studies DeepSeekCoder 7B/33B and GPT-family references. LLM-SR (ICLR 2025) compares Mixtral and GPT-3.5, adding GPT-4o on a subset with equal iterations. LLM-SRBench (ICML 2025) uses Llama-3.1-8B, GPT-3.5 and GPT-4o-mini with a common call budget. These motivate SemZip’s diversity/control axes but are not log-compression baselines or evidence that six models are mandatory.

24 Historical single-synthesis gated replacement follow-up

All seven prior evolution datasets with at least five original 100,000-line blocks enter this cohort. The first ten original blocks are used, or the complete shorter file. Tails are unchanged. Block 0 supplies the exact saved initial program and policy. The protocol was recorded before the campaign began. It permits one new candidate per triggered dataset and preserves every rejection. These results are a developmental follow-up, not a replication or component-wise ablation of the older nine-dataset protocol.

Blocks 1 and 2 must each contain at least ten eligible numeric residual groups, or each have archive/raw byte fraction at least 1.10 times block 0. Eligibility requires fifty supporting lines and two distinct values within that block. Groups are reset between blocks for this trigger. A trigger invokes fresh whole-plan GPT-4o resynthesis from block 2 using the existing generic sampler with a twenty-call ceiling, temperature zero, a 16,384-token request ceiling, and zero retries. No old-tag admission restriction is applied. Duplicate candidate tags are rejected as ambiguous.

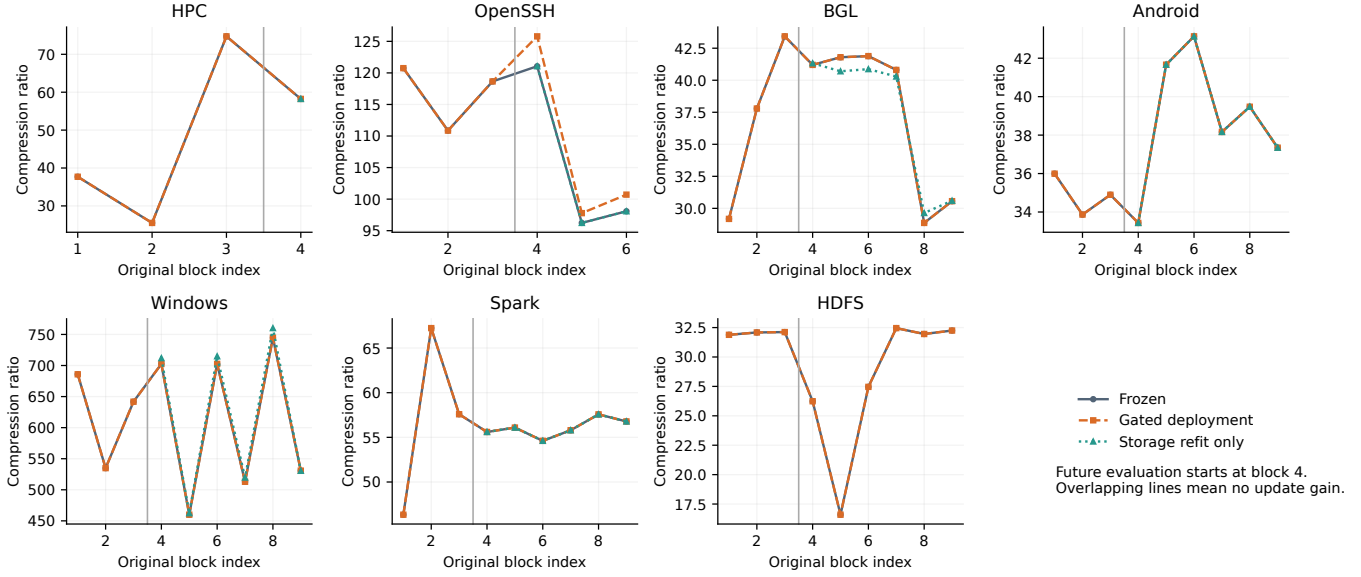
Candidate storage is fitted on block 2. The unchanged program is also fitted on block 2 to obtain a storage-only control. Independent block 3 is used only for validation. The candidate must reconstruct exactly and reduce complete archive bytes by at least 1% against both the original frozen deployment and the storage control. Passing candidates undergo three interleaved encode measurements with the original deployment, and their median time must be at most 1.20 times the old median. This permits a slowdown of up to 20%. Publication occurs after block 3 and is immutable for future blocks 4 onward. An encountered runtime failure is retained and uses the original archive as fallback. Attempted work is not free.



Dataset	Outcome	Blocks	Calls	Frozen future	Deployed future
HPC	Size rejected	5	3	58.227	58.227
OpenSSH	Published	7	9	105.016	107.772
BGL	Size rejected	10	2	36.668	36.668
Android	Size rejected	10	3	38.641	38.641
Windows	Size rejected	10	19	588.978	588.978
Spark	Size rejected	10	9	56.061	56.061
HDFS	No trigger	10	0	26.432	26.432

The gated follow-up triggers on 6/7 datasets and publishes 1 replacement plan. Future savings are positive in 1 case and negative in 0. All remaining outcomes are unchanged.

25 Historical single-synthesis gated follow-up trajectories



The vertical line separates validation block 3 from future blocks. All published positive, zero, and negative changes are retained. No publication means that the deployed trace coincides with the original. The storage-only control is separately measured on future blocks when its fit succeeds, but it is never automatically deployed. Ratio movement across blocks reflects input composition as well as any method change. Compare paired methods within each block.

Dataset	Future blocks	Frozen bytes	Deployed bytes	Control bytes	Saving (%)
HPC	1	79739	79739	79739	+0.000
OpenSSH	3	273629	266633	273629	+2.557
BGL	6	2359686	2359686	2368058	+0.000
Android	6	1939054	1939054	1939054	+0.000
Windows	6	248698	248698	246090	+0.000
Spark	6	1078450	1078450	1078450	+0.000
HDFS	6	3187488	3187488	—	+0.000

Complete archive sizes include semantic data, residual data, and metadata. Future aggregate ratios divide summed raw bytes by summed archive bytes. Independent archive-only reconstruction checks every deployed original block with an intentionally missing external storage policy. Fixed policy decisions are checked for every encoded block. The source runtime and backend are unchanged from the matched-backbone study. Compression and decoding launch a fresh CLI for each block, with block-local dictionaries, difference state, and file aliases. The historical residual observer reuses a parent runtime between observed blocks without the new cache reset.

A separate zero-API control replays the exact fourteen observations (blocks 1–2 of all seven datasets) with explicit entry/exit clearing of input-derived caches and mutable compiled callables. Every coverage field, eligible-group count, and complete residual-group object matches its original record, and all seven novelty, size, and combined trigger decisions are unchanged. Original input, plan, decision, and source hashes remain unchanged. The control performs no encoding, decoding, storage fitting, or candidate generation. Its diagnostic execution time is not historical monitoring overhead. This finite equality check addresses these fourteen observations without retroactively upgrading the historical campaign to the current strict process protocol or proving cache independence on other inputs.

26 Historical single-synthesis gated follow-up triggers, validation, and costs

Dataset	Trigger	Groups 1/2	Synthesis s	Fit+generation s	Gate s
HPC	size	24/7	12.00	18.91	5.82
OpenSSH	size	3/4	84.32	149.23	107.64
BGL	novelty	26/17	12.51	39.78	18.02
Android	novelty	300/333	55.88	98.48	21.56
Windows	novelty	11/13	87.03	144.81	24.97
Spark	novelty	70/37	58.24	103.33	22.03
HDFS	none	9/9	0.00	0.00	0.00

Fit+generation includes storage-control fitting, synthesis, candidate fitting, and a full training-block encode/decode check. Synthesis seconds are a subset, not additive. Gate time includes candidate/control validation and extra speed checks when applicable, while the ordinary frozen validation encode is outside this interval. These are wall durations on a shared developer host, not isolated latency estimates or monetary costs. Trigger-monitor overhead is not separately timed, and encoding timings exclude that monitoring, offline work, imports and standalone decoding. No end-to-end production speedup or economic break-even is established.

Dataset	Frozen valid.	Control valid.	Candidate valid.	Outcome
HPC	120119	120119	120119	Size rejected
OpenSSH	98211	98211	95483	Published
BGL	322959	321687	365423	Size rejected
Android	344355	344355	358167	Size rejected
Windows	38007	37503	37163	Size rejected
Spark	174849	174849	177929	Size rejected
HDFS	—	—	—	No trigger

Missing validation entries mean the candidate did not reach that stage, not a zero-byte archive. The evidence ledger retains full exceptions, original candidate plans, and API request/response records. Training or validation failures are not replaced by fresh generations. Synthesis calls, failures, and unchanged outcomes stay in the experiment denominator.

The trigger reduces when training is attempted but does not predict success with an established probability. A validation gate measures a candidate on already arrived independent data, while future blocks provide the efficacy test. Reporting only successful publications would conceal false acceptance, regressions, and wasted attempts. Rejected candidates are not counterfactually deployed on future blocks, so false-negative rates are unknown. Unchanged outcomes support retaining the initial program in these measured windows, not universal sufficiency of initial training under production drift.

Future candidates have 0 runtime failures. Failed-attempt wall time is 0.000 seconds, separate from successful encoding. All 62 deployed original blocks pass archive-only reconstruction.